

# 7 The Application Layer

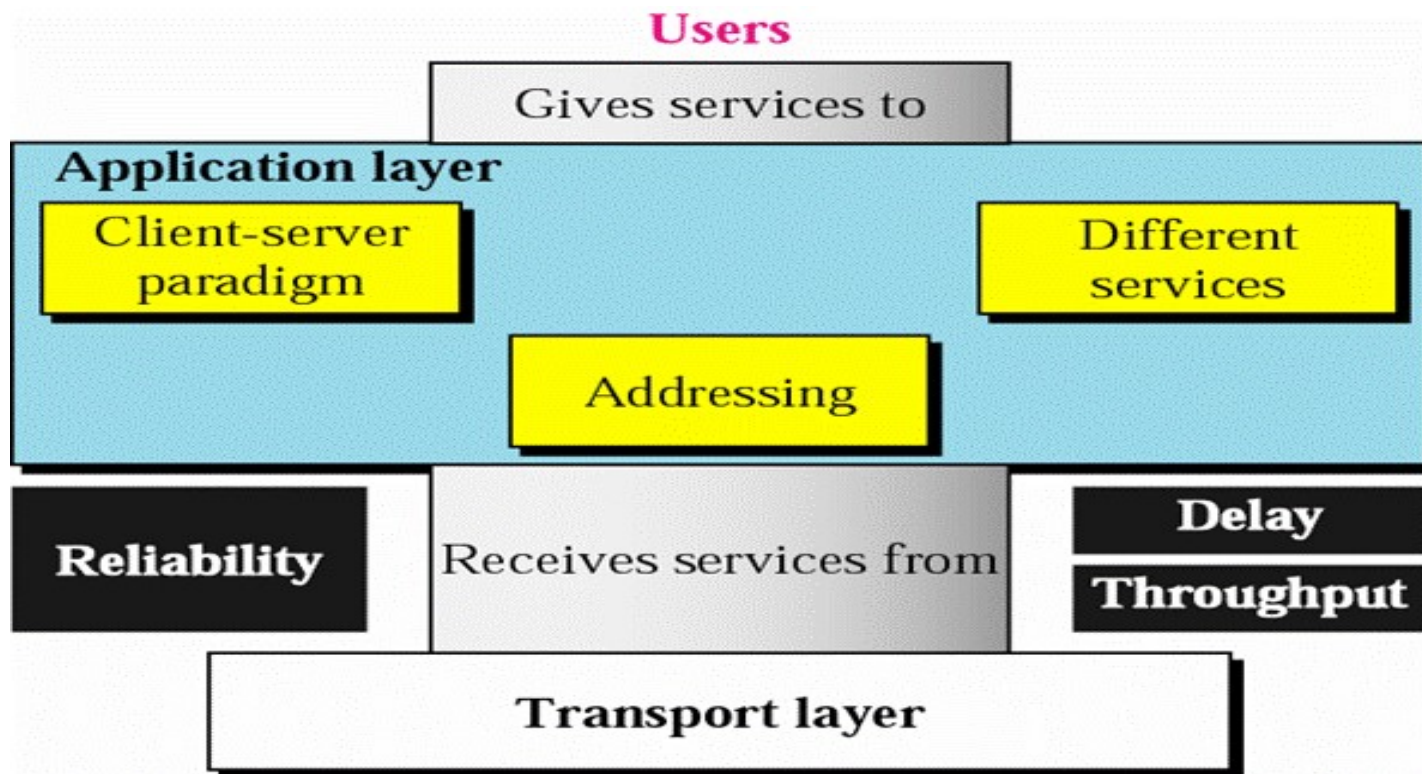
(about 3-4 hours)

# Outline

- Introduction to Application Layer
  - ◆ Positions and functions of application layer
  - ◆ Peer-to-peer paradigm vs. Client-server paradigm
- Domain Name System (DNS)
- Email System and Protocols
- World Wide Web (WWW)

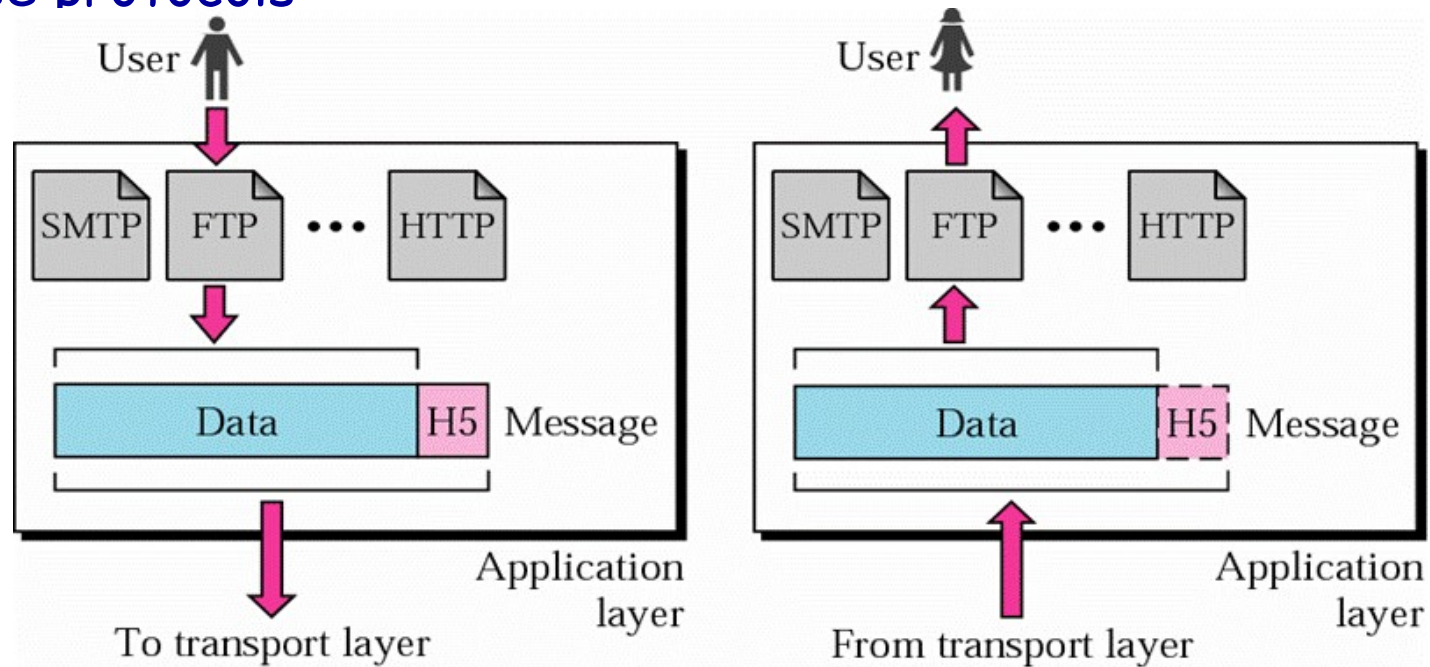
# Position of Application Layer

## ■ Top layer of the Layering Architecture



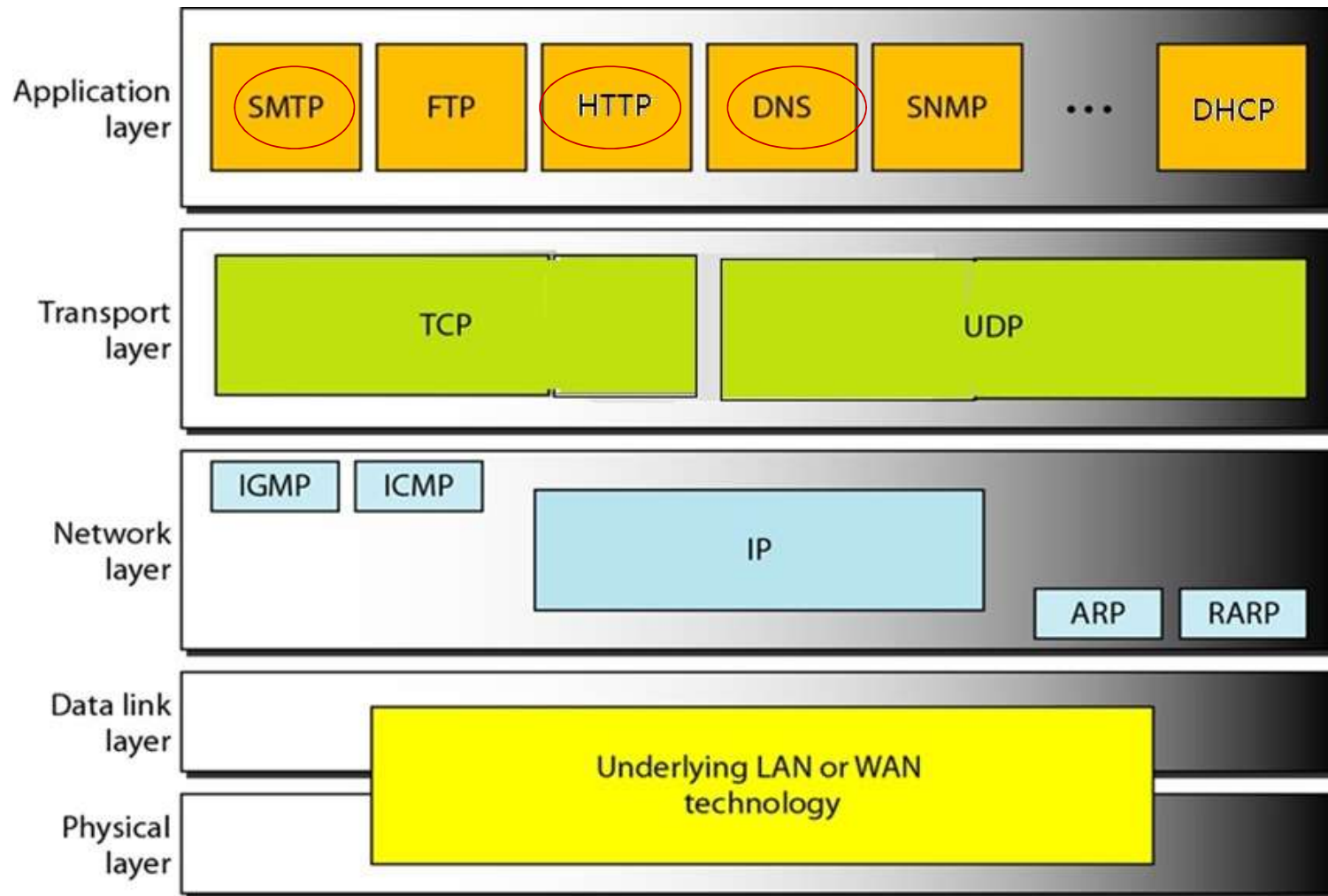
# Duties and Features of Application Layer

- Providing various applications to end users
- Maximum number of protocols and most complex
- Each protocol is specified to a type of application, no general-purpose protocols



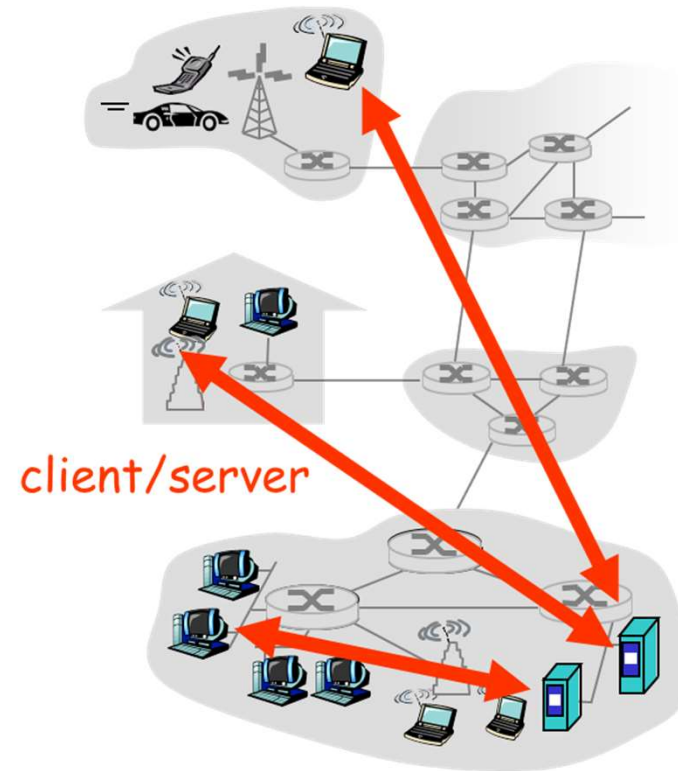
[Forouzan]

# Conventional Internet Application Protocols



# Client-server Paradigm

- The tasks, capabilities and protocol machines are different at each side.
- server:
  - ◆ always-on
  - ◆ permanent IP address
  - ◆ server farms for scaling
- clients:
  - ◆ communicate with server
  - ◆ may be intermittently connected
  - ◆ may have dynamic IP addresses
  - ◆ do not communicate directly with each other

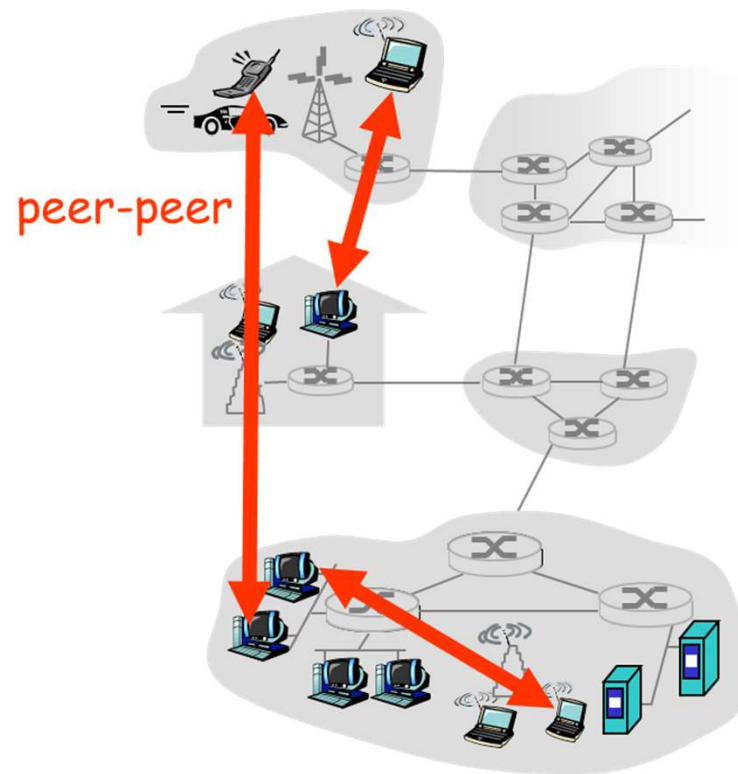


Forms the basis of computer networks

[Kurose]

# Pure P2P Architecture in Computer Networks

- no always-on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses



[Kurose]

# Outline

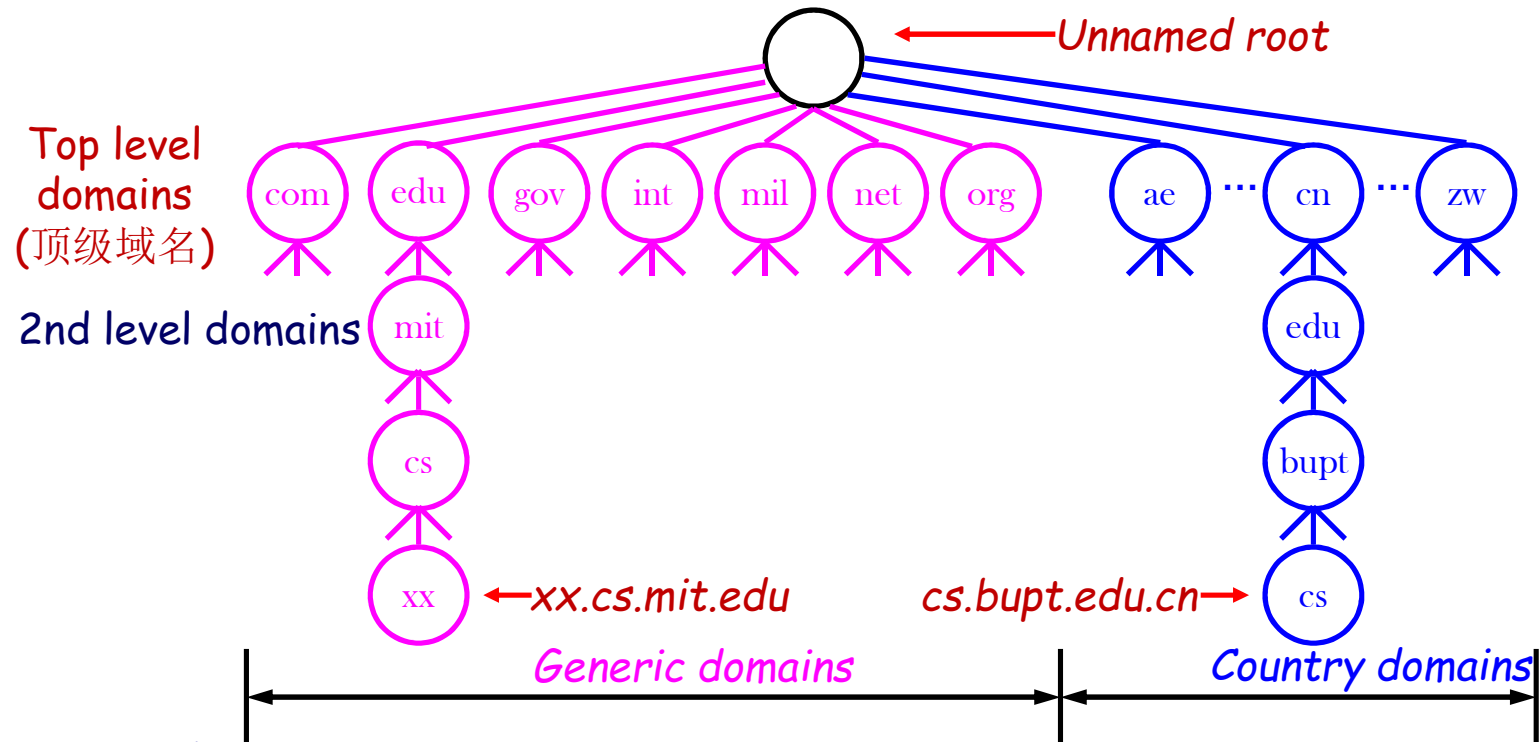
- Introduction to Application Layer
- Domain Name System (DNS) (Textbook 7.1)
  - ◆ Hierarchical namespace
  - ◆ Resource records
  - ◆ Name servers
  - ◆ Name Resolution Procedure
- Email System and Protocols
- World Wide Web (WWW)



# DNS: Domain Name System(域名系统)

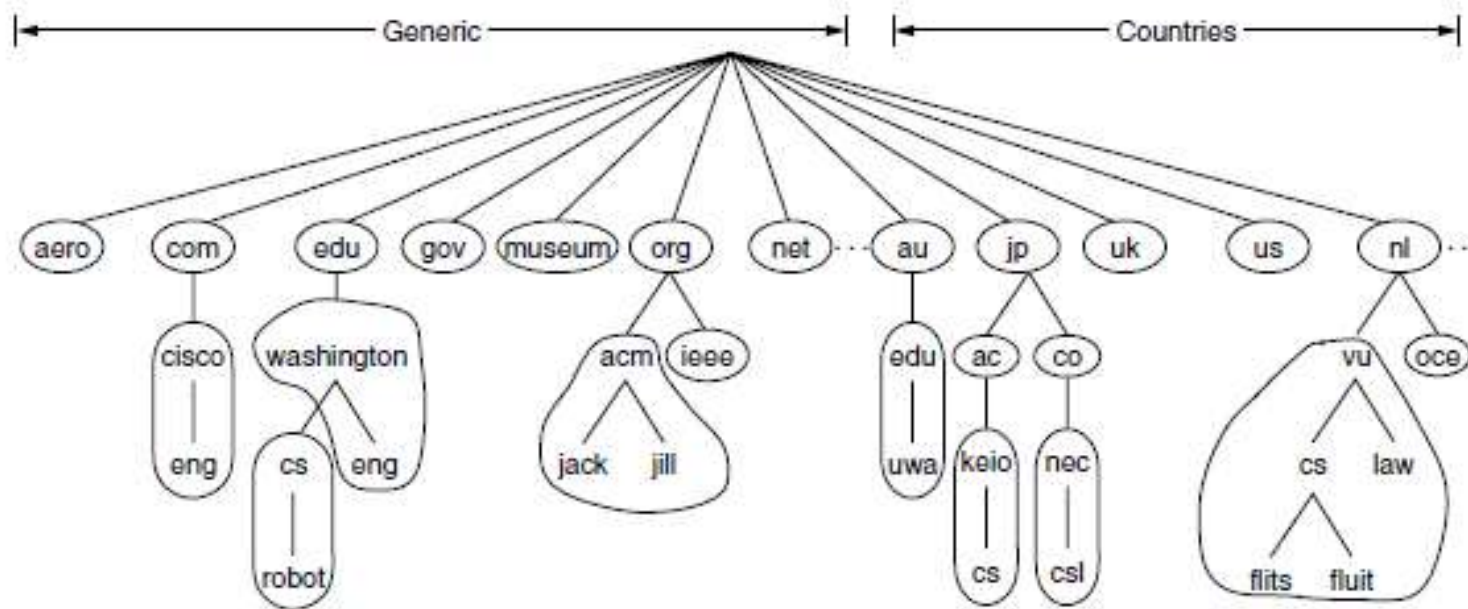
- A client-server application that identifies each host on the Internet with a unique user-friendly name(domain name),
  - ◆ e.g. *www.bupt.edu.cn* instead of 123.127.134.10
- Providing support for other applications
- Translating domain name (host name) to IP address
- Out-dated "hosts.txt"
- Features
  - ◆ *Hierarchical namespace*: Host have a composite names that are all hierarchically organized
    - In contrast to *Flat Name Space*: Each host in the network is unique and independent from each other
  - ◆ Distributed database

# Hierarchical Namespace



- Within an organization,
  - ◆ Subdivision possible
  - ◆ Arbitrary levels possible (maximum depth =128)
  - ◆ Not standardized, controlled locally by the organization
- Naming policy: by the path upward from leaf to root, separated by "."

## Zones(☒)(Figure 7-5)



- A **zone** corresponds to an administrative authority that is responsible for that portion of the hierarchy
- Eg. **bupt** controls **x.bupt.edu.cn**

## Resource Records(资源记录)

- Each domain in the DNS has one or more Resource Records (RRs)
- Each RR has the following information
  - ◆ Owner: the domain name
  - ◆ Type: specifies the type of the resource in this RR
    - SOA – Start Of Authority, identifies the zone
    - A – Host Address
    - MX – Mail Exchanger
    - CNAME – Canonical Name, real name used for management
    - ...
  - ◆ Class: specifies the protocol family to use
    - IN – the Internet system
  - ◆ TTL: specifies the Time To Live (in unit of second) of the cached RRs

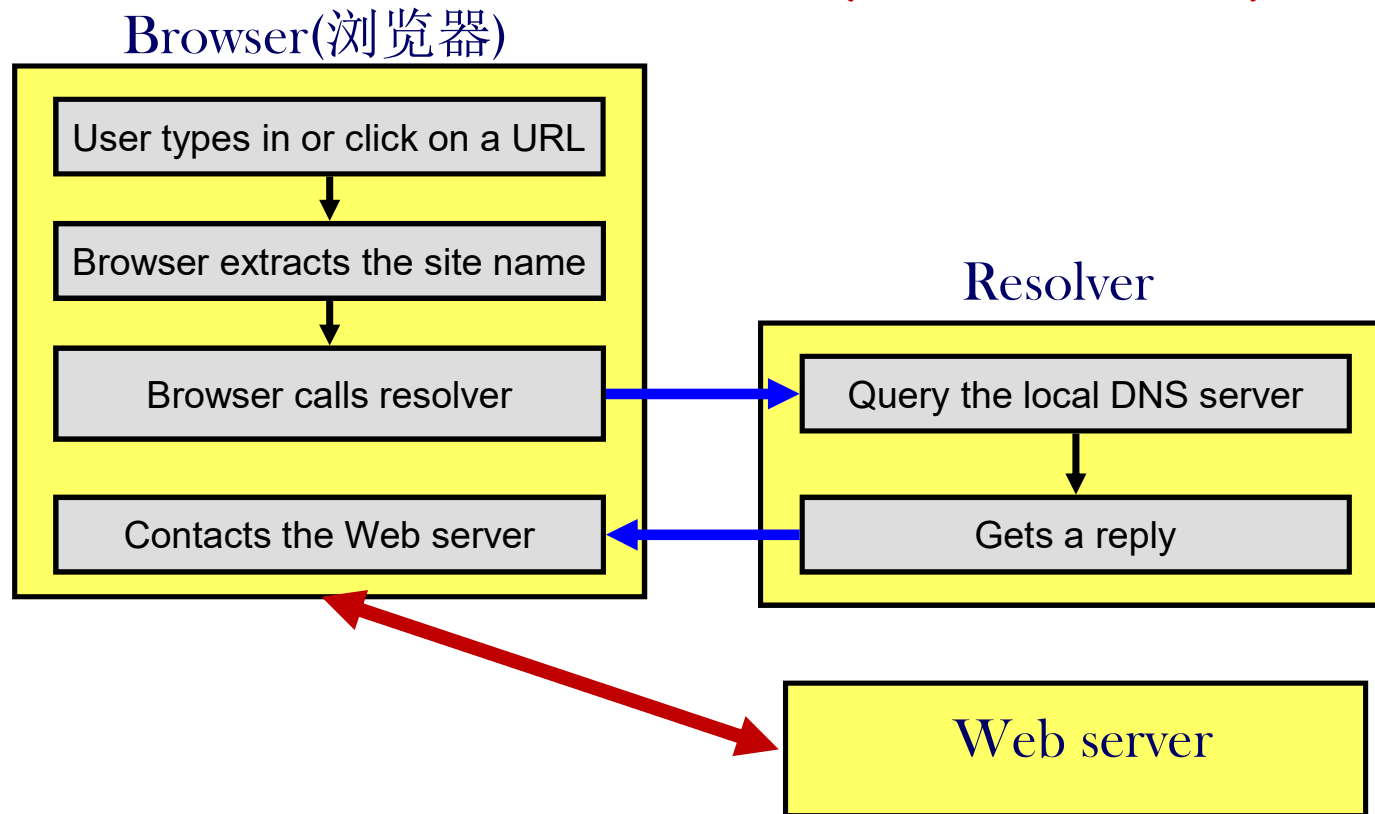
# Sample of DNS Database

| Domain name                       | Time-to-live | Class | Type  | Value                                   |
|-----------------------------------|--------------|-------|-------|-----------------------------------------|
| ; Authoritative data for cs.vu.nl |              |       |       |                                         |
| cs.vu.nl.                         | 86400        | IN    | SOA   | star boss (9527,7200,7200,241920,86400) |
| cs.vu.nl.                         | 86400        | IN    | MX    | 1 zephyr                                |
| cs.vu.nl.                         | 86400        | IN    | MX    | 2 top                                   |
| cs.vu.nl.                         | 86400        | IN    | NS    | star                                    |
| star                              | 86400        | IN    | A     | 130.37.56.205                           |
| zephyr                            | 86400        | IN    | A     | 130.37.20.10                            |
| top                               | 86400        | IN    | A     | 130.37.20.11                            |
| www                               | 86400        | IN    | CNAME | star.cs.vu.nl                           |
| ftp                               | 86400        | IN    | CNAME | zephyr.cs.vu.nl                         |
| flits                             | 86400        | IN    | A     | 130.37.16.112                           |
| flits                             | 86400        | IN    | A     | 192.31.231.165                          |
| flits                             | 86400        | IN    | MX    | 1 flits                                 |
| flits                             | 86400        | IN    | MX    | 2 zephyr                                |
| flits                             | 86400        | IN    | MX    | 3 top                                   |
| rowboat                           |              | IN    | A     | 130.37.56.201                           |
|                                   |              | IN    | MX    | 1 rowboat                               |
|                                   |              | IN    | MX    | 2 zephyr                                |
| little-sister                     |              | IN    | A     | 130.37.62.23                            |
| laserjet                          |              | IN    | A     | 192.31.231.216                          |

# DNS Client

- Known as **resolver**(解析器)

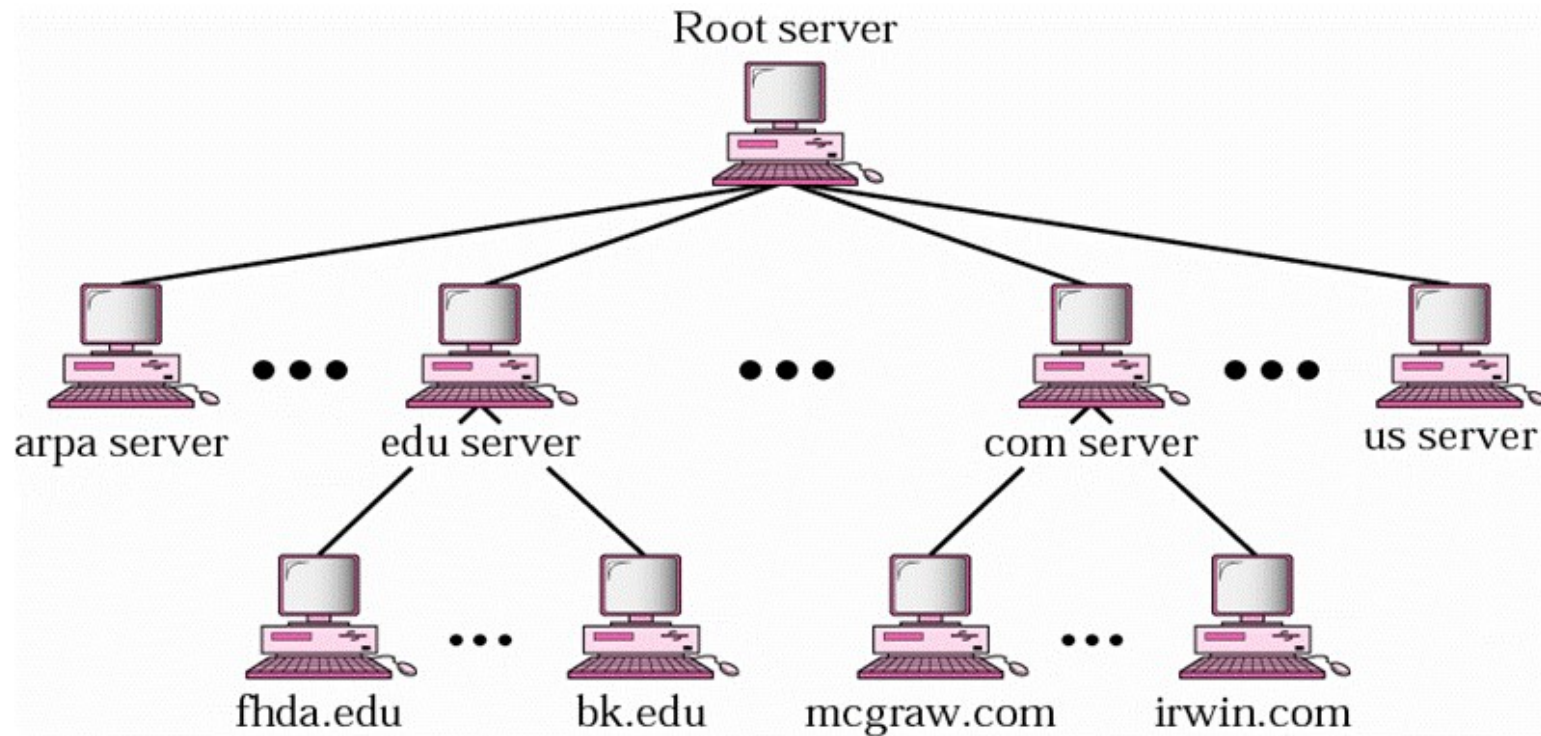
- ◆ software running on client, able to access at least one name server, i.e. **Local name server**(本地名字服务器)



# DNS Servers

- Multiple DNS servers used
  - ◆ Arranged in hierarchy
  - ◆ Each server has authority over a portion of the hierarchy:  
**Authoritative Name Server**(权威名字服务器)
  - ◆ Each server contains all the records for the hosts in its **zone**
  - ◆ How does a server know about other servers that are responsible for the other zones?
    - Every server knows the root
    - Root server knows about all TLD servers
    - Every server knows the server(s) further down the hierarchy

# Hierarchy of Name Servers



*[Forouzan]*



# Primary and Secondary Servers

- DNS uses backup server(s)
- **Primary server**: creates, maintains, and updates information about its zone
- **Secondary server**: gets its information from a primary server; as a backup server in case of failure
- Both servers have **authority** over their zone
- ISPs
  - ◆ Offering DNS service to subscribers
- Small organizations and individuals
  - ◆ Only need domain names for computers running servers
  - ◆ Contract with an ISP for domain service

# DNS Lookup

## ■ Applications

- ◆ Becoming DNS client
- ◆ Sending request first to local name server

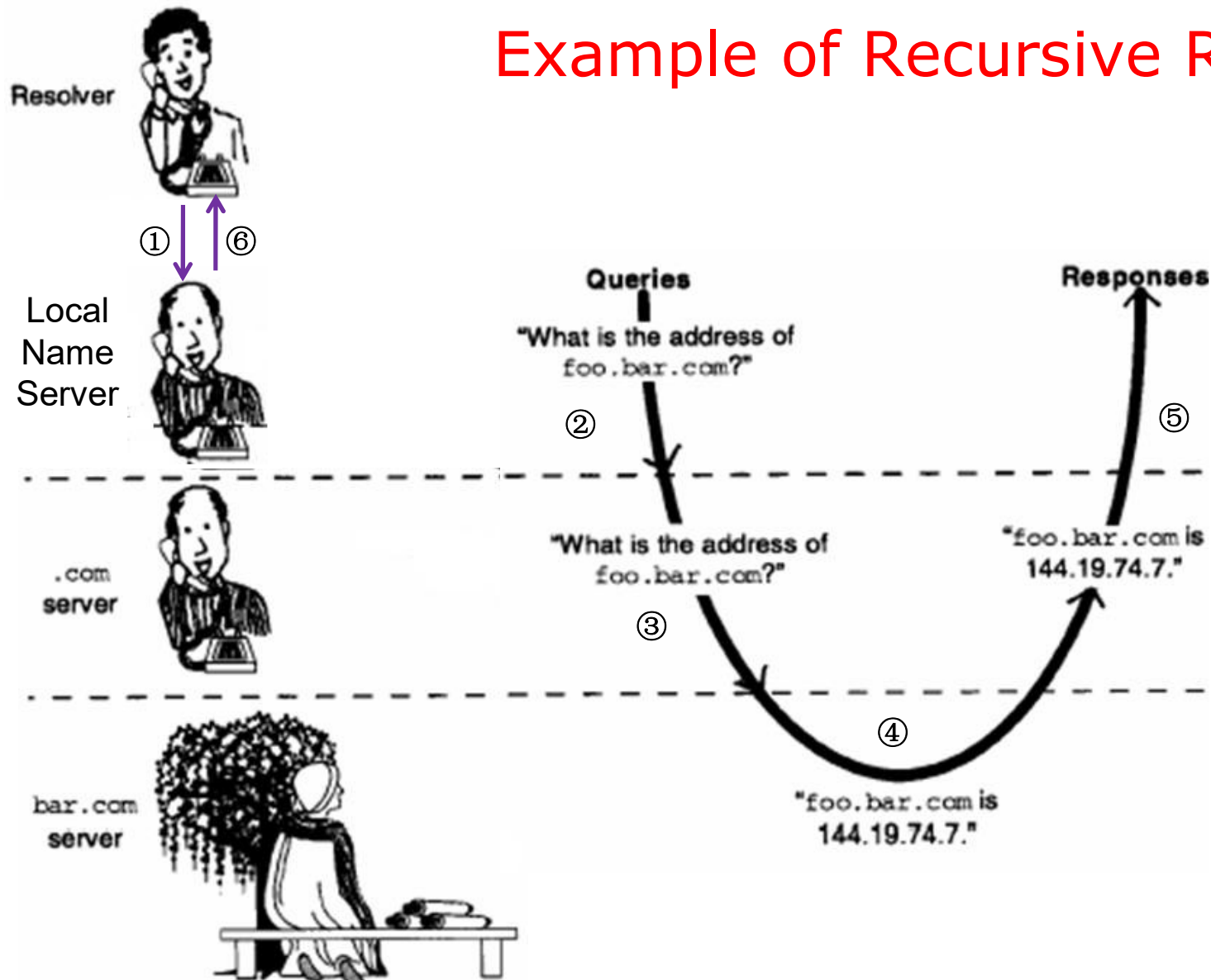
## ■ Local name server

- ◆ Listening at port 53, normally using UDP
- ◆ If answer known, returns response
- ◆ If answer unknown
  - Requests root or TLD name server
  - Follows links
  - Until reaching **Authoritative Name Server**, which performs name translation and returns response

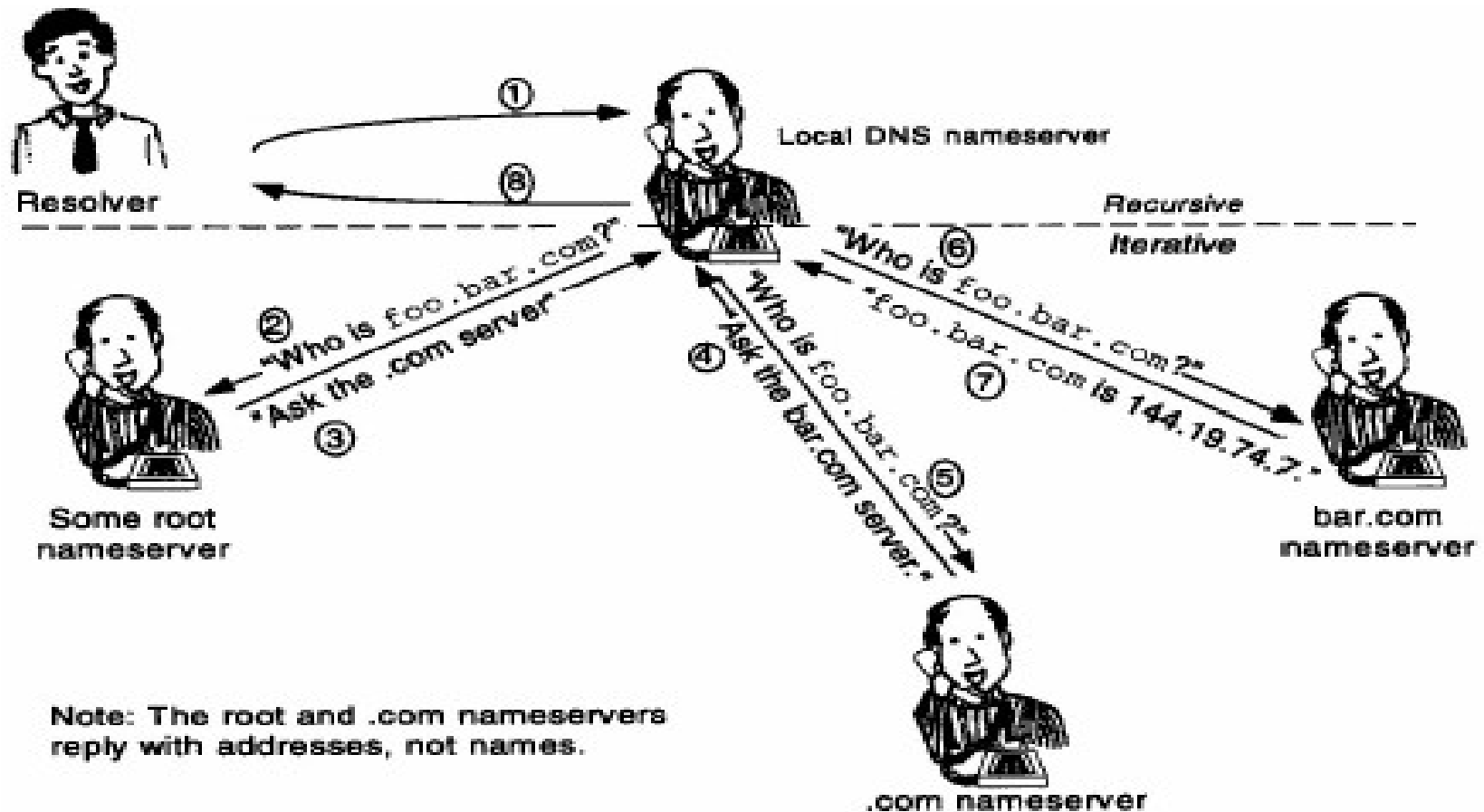
# Name Resolution Methods

- A query is made to a **local name server**. If local name server can not resolve the name, then it will query another server.
- **Recursive resolution(递归解析/递归查询):**
  - ◆ If the **queried server** does not have the information, it must **make** the appropriate **query** or queries to get the information
  - ◆ Generally, a server fulfills a recursive query either with data in its own memory or by making another recursive query
- **Iterative resolution(迭代解析/迭代查询):**
  - ◆ If the queried server does not have the information, it may then respond with the **address** of another server; the **local name server** (on behalf of resolver )then **queries** that server (which might respond with the address of another server, and so on)
  - ◆ Commonly used by name servers on the Internet

# Example of Recursive Resolution

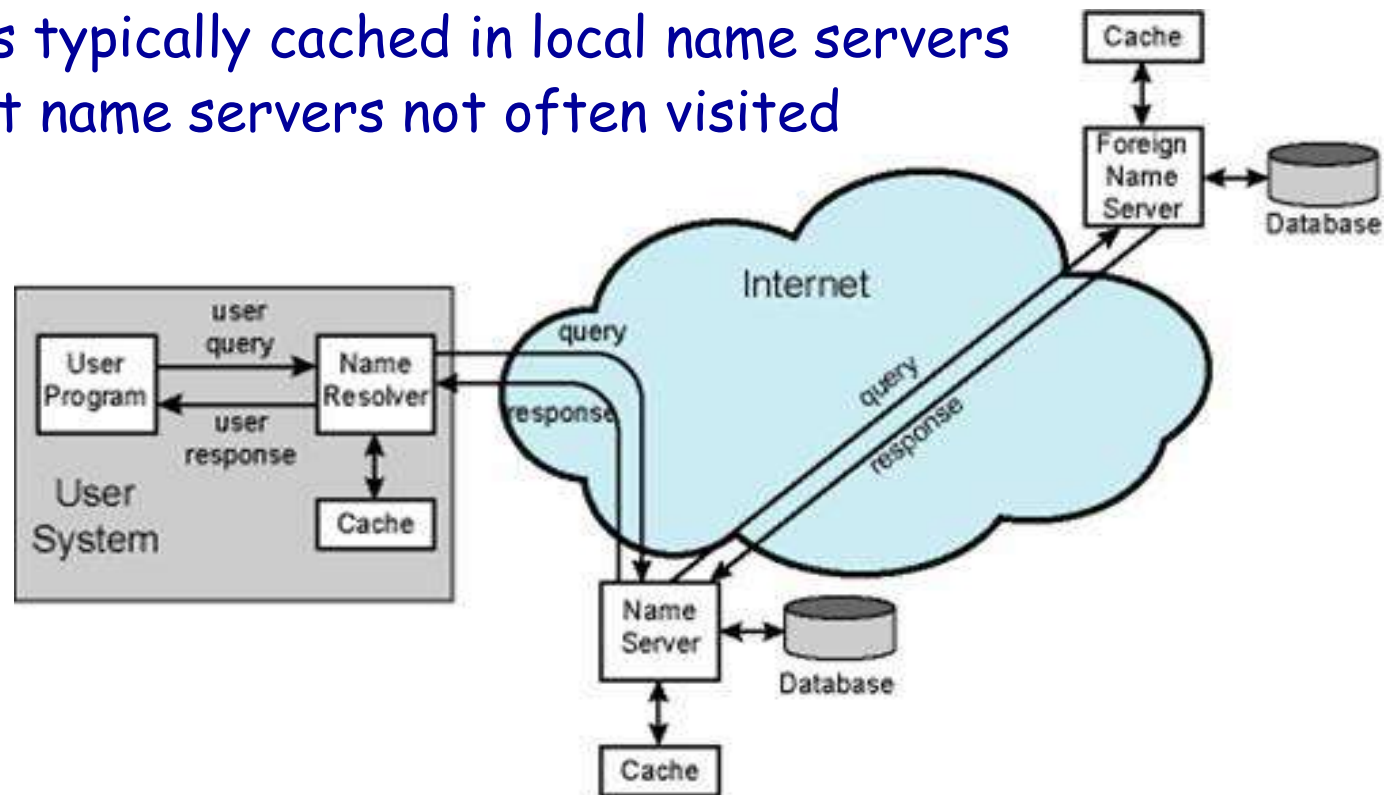


# Example of Iterative Resolution



# DNS Caching

- Server always caches answers
- Host can cache answers
- TLD servers typically cached in local name servers
  - ◆ Thus root name servers not often visited



# DNS Tools: nslookup

## ■ Query name servers iteratively

```
C:\Users\Administrator>nslookup -query=MX bupt.edu.cn
服务器:  UnKnown
Address:  10.3.9.4

非权威应答:
bupt.edu.cn      MX preference = 10, mail exchanger = mxbiz2.qq.com
bupt.edu.cn      MX preference = 5, mail exchanger = mxbiz1.qq.com

C:\Users\Administrator>nslookup mxbiz2.qq.com
服务器:  UnKnown
Address:  10.3.9.4

非权威应答:
名称:    mxbiz2.qq.com
Addresses: 163.177.89.176
           61.241.49.98

C:\Users\Administrator>
```

# DNS tools: dig

```
student@BUPTIA:~$ dig www.bupt.edu.cn

; <<>> DiG 9.9.5-3ubuntu0.8-Ubuntu <<>> www.bupt.edu.cn
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5439
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;www.bupt.edu.cn.                IN      A

;; ANSWER SECTION:
www.bupt.edu.cn.                2796    IN      CNAME   vn.bupt.edu.cn.
vn.bupt.edu.cn.                 68      IN      A       10.3.9.161
```



## DNS小结

- 应用层：面向用户，每个协议支持一类应用
- 两种典型的应用模式：**C/S模式**和**P2P模式**
- **DNS目标**：**实现域名和IP地址之间的映射(转换)**
  - ◆ 分层的域名空间、分布式数据库
  - ◆ 域名的规定
  - ◆ **Client(解析器)**请求**本地域名服务器**（UDP， 端口**53**）
  - ◆ 本地名字服务器请求其他服务器(最终到权威域名服务器)
    - 迭代解析（因特网普遍采用）
    - 递归解析
- **DNS目标很简单，但实现系统庞大而复杂**
- 具有**高效率(缓存机制)**和**可靠性(镜像、辅助域名服务器)**
- 安全性：**DNSSEC**

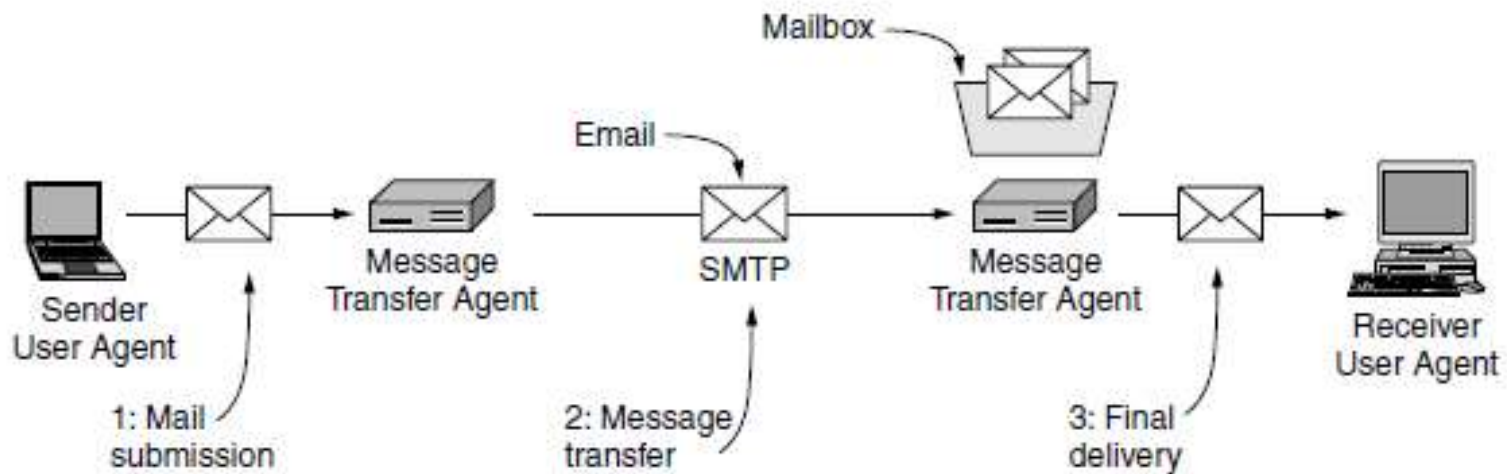
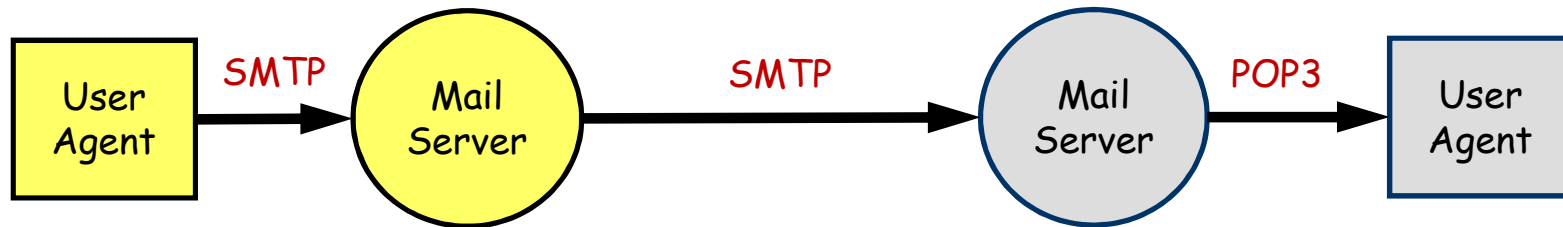
# Outline

- Introduction to Application Layer
- Domain Name System (DNS)
- Email System and Protocols
  - ◆ System components
  - ◆ Message Format
  - ◆ SMTP
  - ◆ POP3
- World Wide Web (WWW)

# What is Email?

- Electronic Mail (email, e-mail)
- Provides a means to send electronic messages from one person to another **asynchronously**
- One of the most popular applications on the Internet and one of the most important communication methods today
- Email service can be provided by
  - ◆ ISPs: @126.com, @163.com, @sina.com, yahoo.com.cn, ...
  - ◆ Corporations and institutes: @baidu.com, @bupt.edu.cn, @ietf.org, ...
  - ◆ Bundled with other services: @139.com, @qq.com, ...

# Components Of Email System (1)



## Components Of Email System (2)

### ■ UA (User Agent)

- ◆ end-user mail program
- ◆ Interface between the end users and the email servers
- ◆ E.g., outlook, foxmail, ...

### ■ Mail Server

- ◆ Responsible for transmitting/receiving emails and reporting status information about mail transferring to the mail sender
- ◆ Both a client and a server

### ■ Email protocols

- ◆ SMTP: used for sending an email
- ◆ POP3: used for receiving an email

# Basic Functions Of Email System

- **Composition** - refers to the process of creating messages and answers.
- **Transfer** - refers to moving messages from the originator to the recipient.
- **Reporting** - refers to the process of informing the originator what happened to the message.
- **Displaying** - means of showing the messages.
- **Disposition** - refers to what happened to the message after it has been read by the receiver.

# Email Address

- The destination address must be in a format that the user agent can deal with
- Many user agents expect DNS addresses of the form *mailboxname@domain* ( RFC 5322 )
- Each email address is *unique* on the Internet because
  - ◆ *Domain* name is unique on the Internet
  - ◆ *Mailboxname* is unique in the domain

# Internet Message Format(RFC 5322)

## ■ Message envelop

- ◆ Contains whatever information is needed to accomplish transmission and delivery
- ◆ Constructed by Mail Servers

## ■ Message contents: comprise the object to be delivered to the recipient

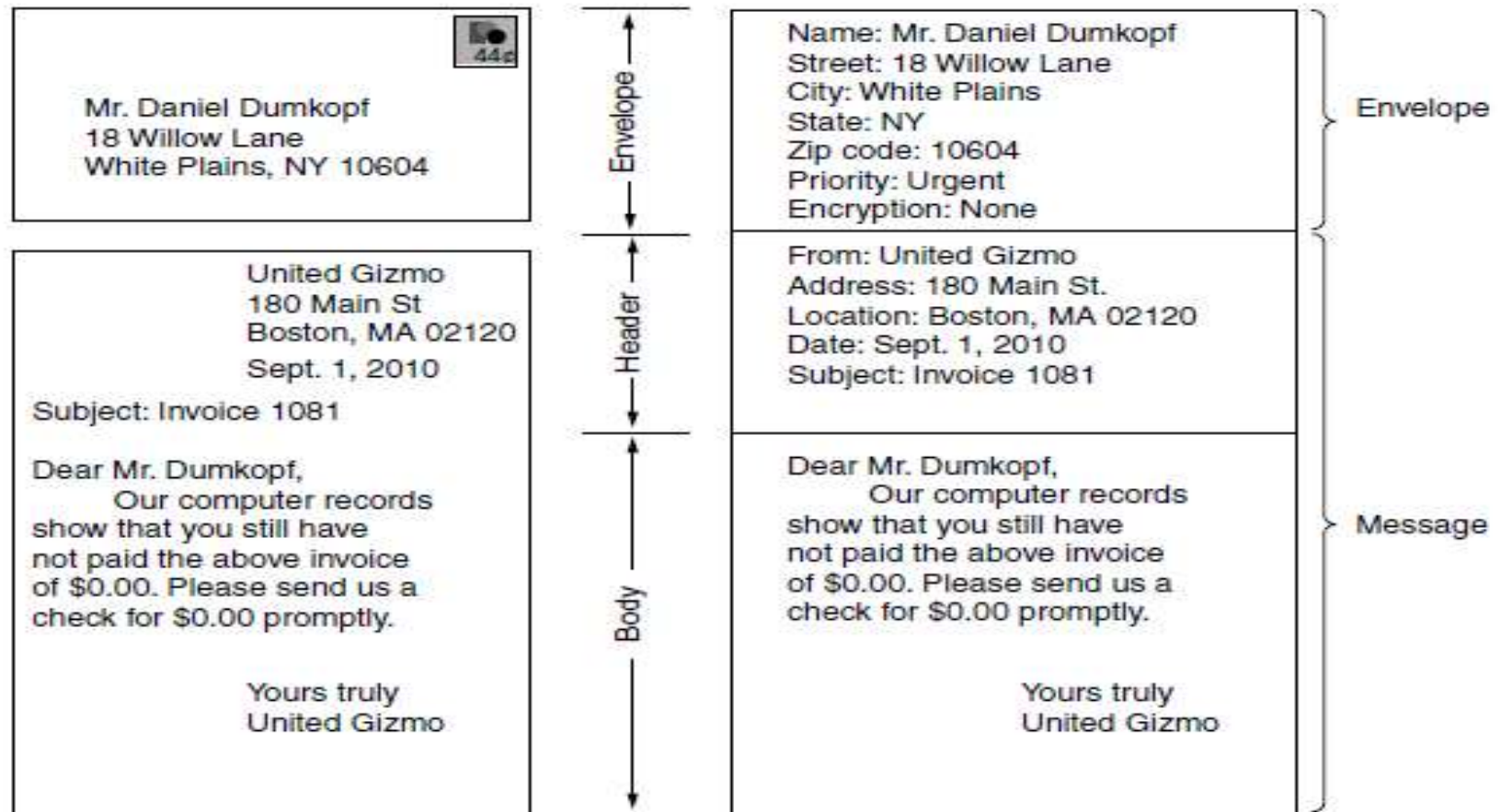
- ◆ Headers: from, to, subject, date, postmarks
- ◆ **Blank line**
- ◆ Body: actual message, may have many parts



# Message Headers Related to Message Transport

| Header       | Meaning                                           |
|--------------|---------------------------------------------------|
| To:          | Email address(es) of primary recipient(s)         |
| Cc:          | Email address(es) of secondary recipient(s)       |
| Bcc:         | Email address(es) for blind carbon copies         |
| From:        | Person or people who created the message          |
| Sender:      | Email address of the actual sender                |
| Received:    | Line added by each transfer agent along the route |
| Return-Path: | Can be used to identify a path back to the sender |

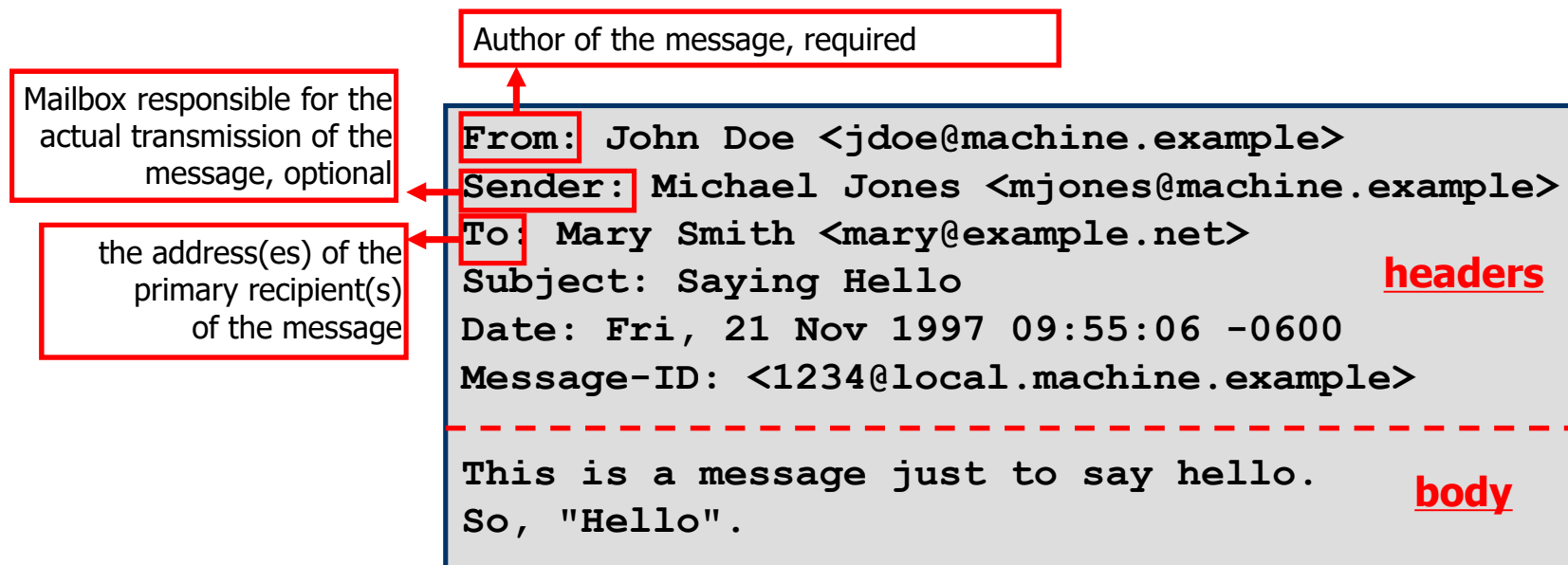
# Paper Mail vs. Email



# Internet Message Format

## —Example RFC 5322 Message

- The **header** part is everything up to the blank line, and the **body** is everything after the blank line



- Problem of RFC 5322: Only 7 bit ASCII format can be used

# MIME

- Multipurpose Internet Mail Extension
- Extension for **multipart & multimedia** email
- Additional mail headers define content
  - ◆ **Type**: text, image, audio, video, application and subtype within (eg text/html, image/gif)
  - ◆ **Encoding** (ASCII , quoted printable, base64) to handle arbitrary binary data when email system can only handle normal ASCII characters
- Supports multipart message content type
  - ◆ each part has its own type and encoding
- MIME messages can be sent using the existing mail programs and protocols
- Widely used now

## MIME: New Headers

| Header                     | Meaning                                              |
|----------------------------|------------------------------------------------------|
| MIME-Version:              | Identifies the MIME version                          |
| Content-Description:       | Human-readable string telling what is in the message |
| Content-Id:                | Unique identifier                                    |
| Content-Transfer-Encoding: | How the body is wrapped for transmission             |
| Content-Type:              | Type and format of the content                       |

## MIME: Content Types and Subtypes

| Type        | Subtype       | Description                                 |
|-------------|---------------|---------------------------------------------|
| Text        | Plain         | Unformatted text                            |
|             | Enriched      | Text including simple formatting commands   |
| Image       | Gif           | Still picture in GIF format                 |
|             | Jpeg          | Still picture in JPEG format                |
| Audio       | Basic         | Audible sound                               |
| Video       | Mpeg          | Movie in MPEG format                        |
| Application | Octet-stream  | An uninterpreted byte sequence              |
|             | Postscript    | A printable document in PostScript          |
| Message     | Rfc822        | A MIME RFC 822 message                      |
|             | Partial       | Message has been split for transmission     |
|             | External-body | Message itself must be fetched over the net |
| Multipart   | Mixed         | Independent parts in the specified order    |
|             | Alternative   | Same message in different formats           |
|             | Parallel      | Parts must be viewed simultaneously         |
|             | Digest        | Each part is a complete RFC 822 message     |

## Example of a Multipart Message(1)

From: alice@cs.washington.edu  
To: bob@ee.uwa.edu.au  
MIME-Version: 1.0  
Message-Id: <0704760941.AA00747@cs.washington.edu>  
Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm  
Subject: Earth orbits sun integral number of times

This is the preamble. The user agent ignores it. Have a nice day.

--qwertyuiopasdfghjklzxcvbnm  
Content-Type: text/html

<p>Happy birthday to you<br>  
Happy birthday to you<br>  
Happy birthday dear <b> Bob </b><br>  
Happy birthday to you</p>

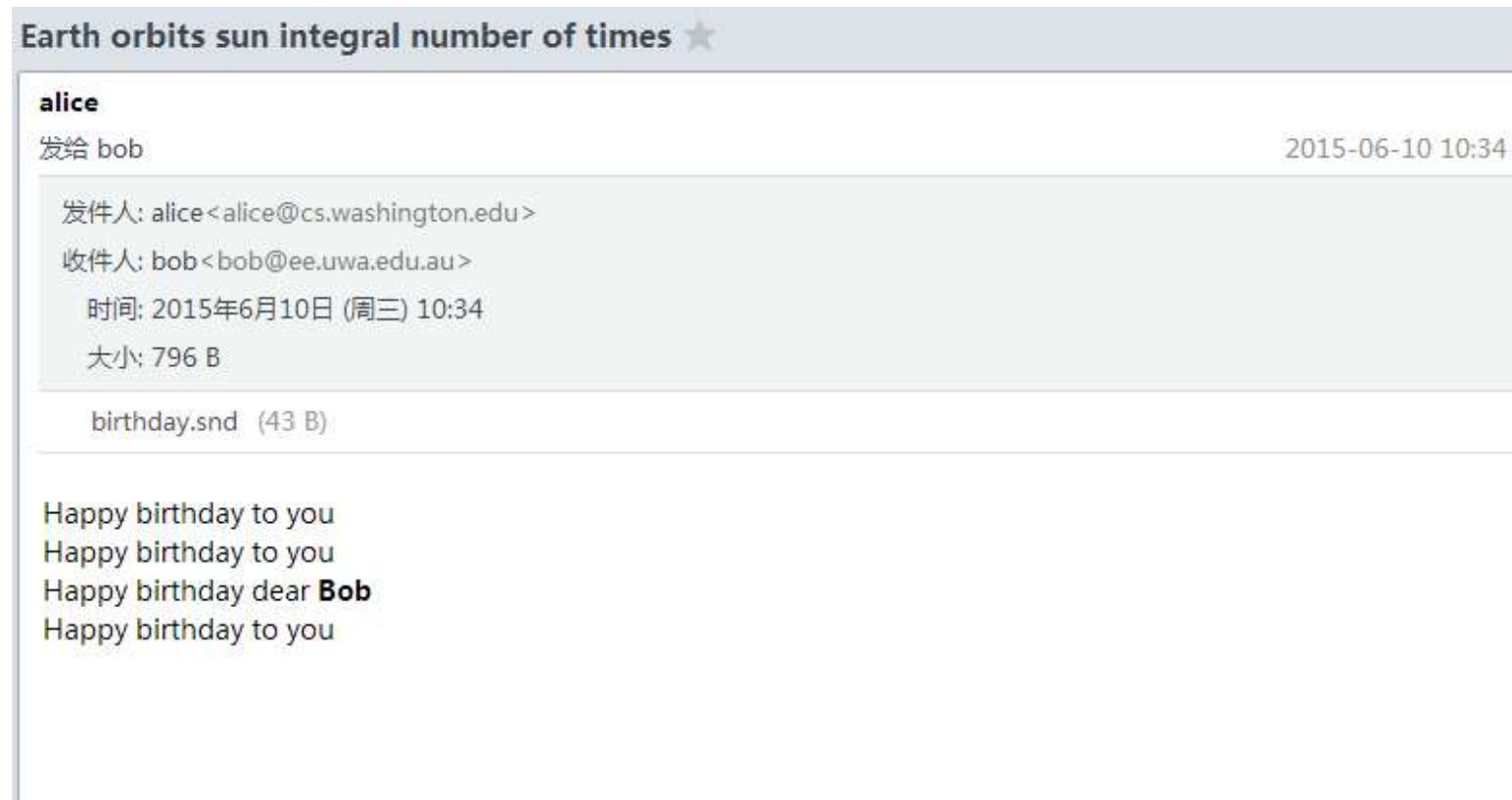
--qwertyuiopasdfghjklzxcvbnm  
Content-Type: message/external-body;  
    access-type="anon-ftp";  
    site="bicycle.cs.washington.edu";  
    directory="pub";  
    name="birthday.snd"

content-type: audio/basic  
content-transfer-encoding: base64  
--qwertyuiopasdfghjklzxcvbnm--



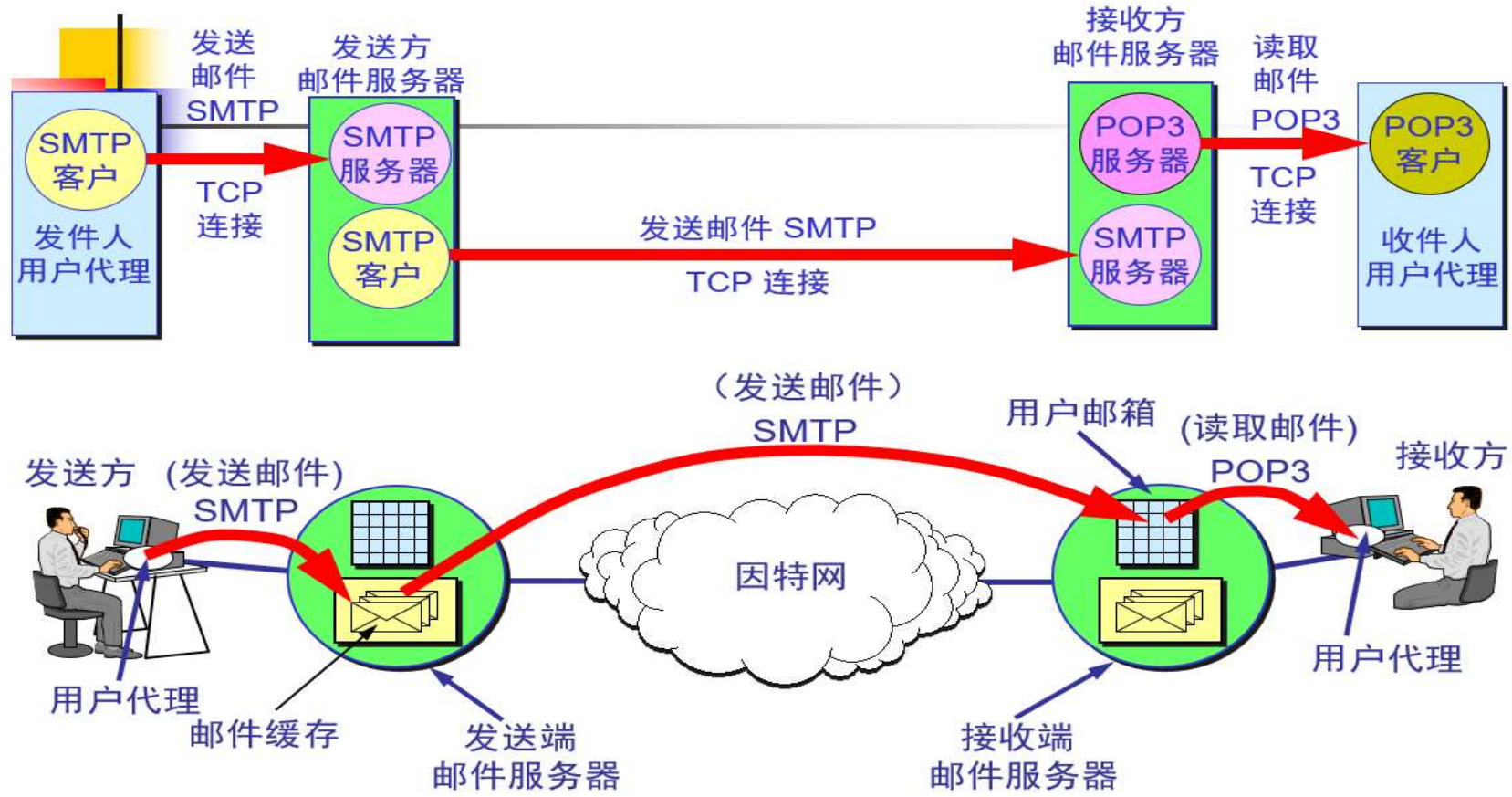
## Example of a Multipart Message(2)

- Saving the above message in \*.eml and opening with Foxmail





# Mail Transfer and Delivery

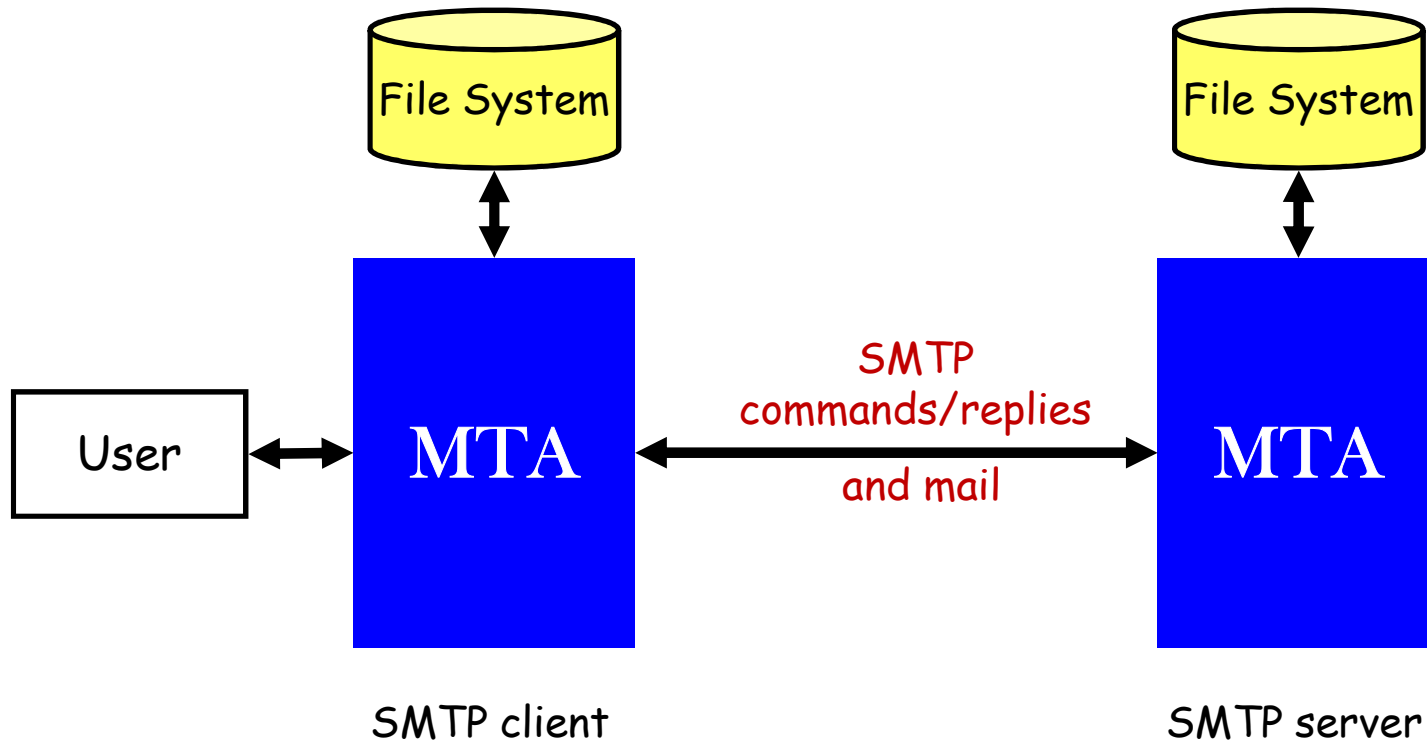


[https://blog.csdn.net/qq\\_40730910](https://blog.csdn.net/qq_40730910)

# SMTP

- Simple Mail Transfer Protocol
- Used in transferring mail messages from UA to mail server, and between mail servers
- SMTP server listens to port 25
- SMTP client establishes a TCP connection to this port
- If the message cannot be delivered, an error report containing the first part of the undeliverable message is returned to the sender – eventually
- SMTP is a simple ASCII protocol

# SMTP Basic Model



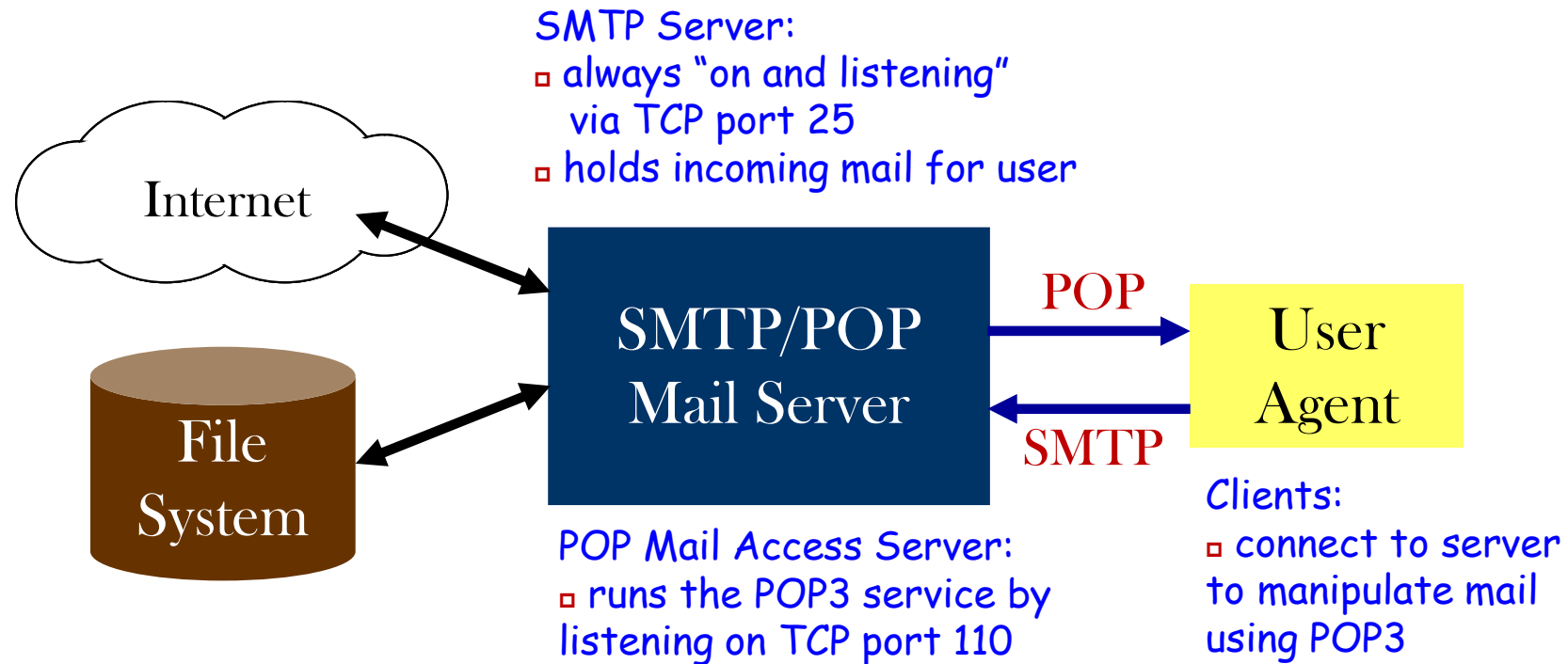
- MTA: Mail Transfer Agent, the mail server using SMTP

## ■ Example of SMTP messages

```
S: 220 ee.uwa.edu.au SMTP service ready
C: HELO abcd.com
S: 250 cs.washington.edu says hello to ee.uwa.edu.au
C: MAIL FROM: <alice@cs.washington.edu>
S: 250 sender ok
C: RCPT TO: <bob@ee.uwa.edu.au>
S: 250 recipient ok
C: DATA
S: 354 Send mail; end with "." on a line by itself
C: From: alice@cs.washington.edu
C: To: bob@ee.uwa.edu.au
C: MIME-Version: 1.0
C: Message-Id: <0704760941.AA00747@ee.uwa.edu.au>
C: Content-Type: multipart/alternative; boundary=qwertyuiopasdfghjklzxcvbnm
C: Subject: Earth orbits sun integral number of times
C:
C: This is the preamble. The user agent ignores it. Have a nice day.
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: text/html
C:
C: <p>Happy birthday to you
C: Happy birthday to you
C: Happy birthday dear <bold> Bob </bold>
C: Happy birthday to you
C:
C: --qwertyuiopasdfghjklzxcvbnm
C: Content-Type: message/external-body;
C:   access-type="anon-ftp";
C:   site="bicycle.cs.washington.edu";
C:   directory="pub";
C:   name="birthday.snd"
C:
C: content-type: audio/basic
C: content-transfer-encoding: base64
C: --qwertyuiopasdfghjklzxcvbnm
C:
S: 250 message accepted
C: QUIT
S: 221 ee.uwa.edu.au closing connection
```

# POP: Basic Model

- Used to transfer mail from a mail server to a UA



# POP: Features

- Essentially store and forward.
  - ◆ Mail is stored on the server until the client connects and then is downloaded to the client.
  - ◆ You *MAY* be able to leave a copy on the server
- Simple protocol and widely used
  - ◆ Similar to SMTP **command/reply** lockstep protocol
  - ◆ Used to retrieve mail for a single user, requires authentication
- Many clients available such as foxmail, outlook
- Common used version: **POP3** (POP Version 3)
- However, very bad for mobile users or users that use multiple machines during the day
  - ◆ Solution: IMAP (Internet Message Access Protocol)

# POP3: Example

```
S: +OK POP3 server ready
C: USER carolyn
S: +OK
C: PASS vegetables
S: +OK login successful
C: LIST
S: 1 2505
S: 2 14302
S: 3 8122
S: .
C: RETR 1
S: (sends message 1)
C: DELE 1
C: RETR 2
S: (sends message 2)
C: DELE 2
C: RETR 3
S: (sends message 3)
C: DELE 3
C: QUIT
S: +OK POP3 server disconnecting
```

# IMAP (Internet Message Access Protocol)

## ■ Features

- ◆ Folders and messages can be stored **either** on the server or on the local computer
- ◆ Since mails can remain on server, it is possible to **access** your same mail store even using a dumb terminal.
- ◆ Much **better for mobile users than POP**
- ◆ Can selectively copy (part of ) messages from the server to the local client based on many criteria

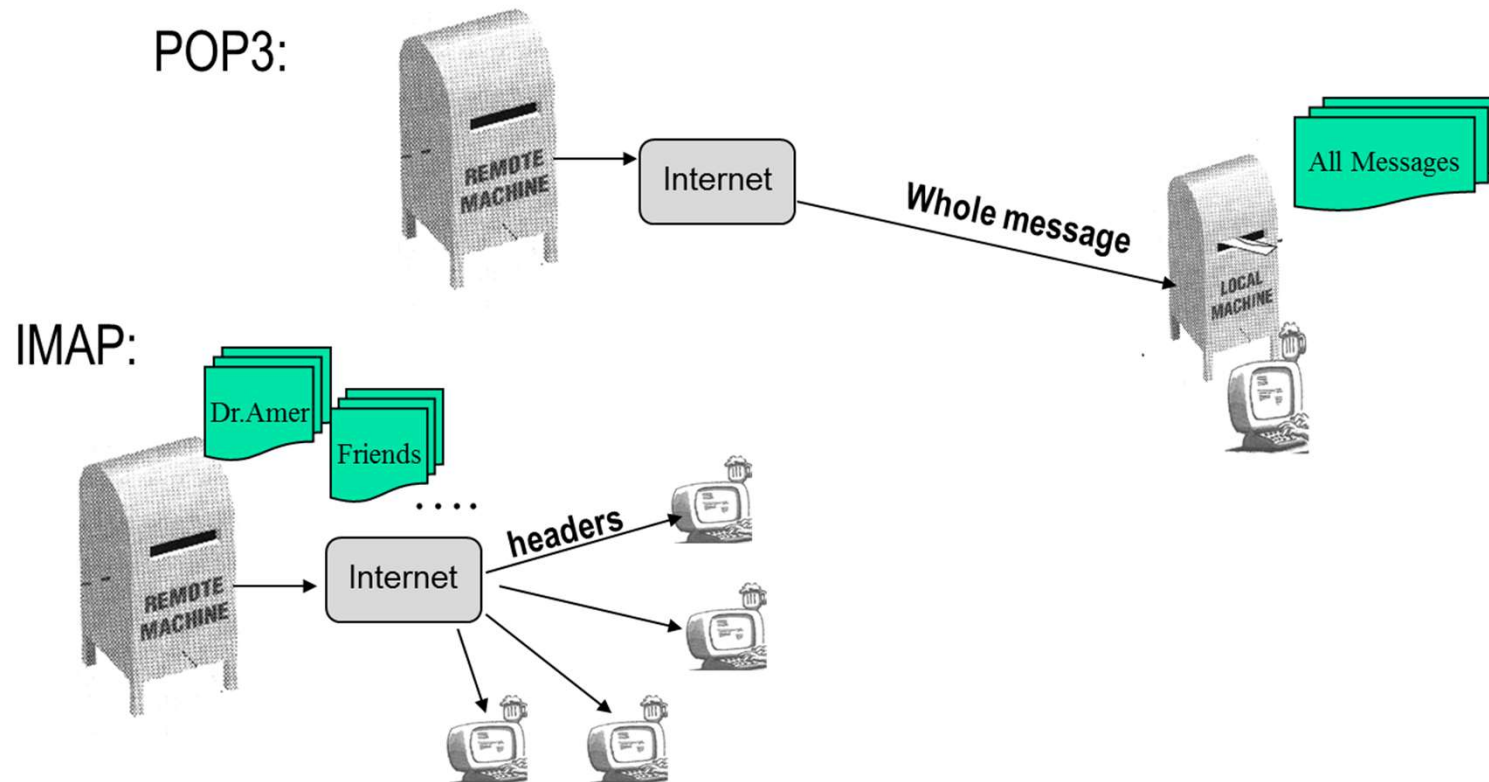
## ■ QQ/163/gmail supports both IMAP and POP3

## ■ Comparison of POP3 and IMAP

- ◆ [https://www.diffen.com/difference/IMAP\\_vs\\_POP3](https://www.diffen.com/difference/IMAP_vs_POP3)



# POP3 vs IMAP(1)

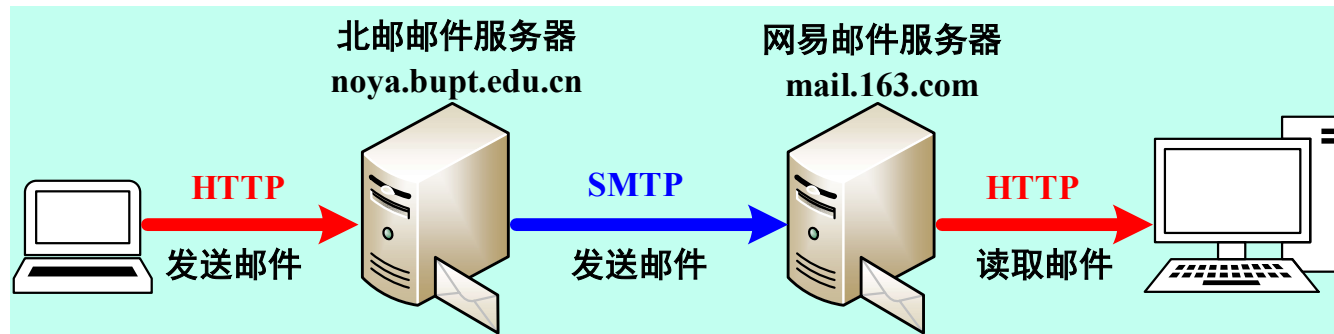


## POP3 vs IMAP

| Feature                        | POP3      | IMAP             |
|--------------------------------|-----------|------------------|
| Where is protocol defined?     | RFC 1939  | RFC 2060         |
| Which TCP port is used?        | 110       | 143              |
| Where is e-mail stored?        | User's PC | Server           |
| Where is e-mail read?          | Off-line  | On-line          |
| Connect time required?         | Little    | Much             |
| Use of server resources?       | Minimal   | Extensive        |
| Multiple mailboxes?            | No        | Yes              |
| Who backs up mailboxes?        | User      | ISP              |
| Good for mobile users?         | No        | Yes              |
| User control over downloading? | Little    | Great            |
| Partial message downloads?     | No        | Yes              |
| Are disk quotas a problem?     | No        | Could be in time |
| Simple to implement?           | Yes       | No               |
| Widespread support?            | Yes       | Growing          |

# Web-based Mail: HTTP

- Can deliver mail message in web page format



厚德博学 敬业乐群

<http://mail.bupt.edu.cn>

The screenshot shows the login interface of the web-based mail system. It features two tabs: '帐号密码登录' (Account Password Login) and '手机验证码' (Mobile Verification Code). The '帐号密码登录' tab is active. Below the tabs, there are input fields for the username (containing 'chengli') and the password (containing '密码'). A checkbox for '5天内自动登录' (Auto-login for 5 days) is present. A large blue button labeled '登录' (Login) is at the bottom. Links for '管理员登录' (Admin Login) and '忘记密码?' (Forgot Password?) are also visible.

# Email系统小结

- 目标：用户之间的异步数据传输
- 邮件格式
  - ◆ 基本邮件格式(IMF)只支持ASCII（英文）
  - ◆ MIME：扩展邮件头，支持多种数据类型、多种语言、邮件内包含多部分(附件)
- 邮件传输协议(Client/Server)
  - ◆ 发送邮件协议：SMTP
  - ◆ 接收邮件协议：POP3, IMAP4(功能更强，更复杂)
- Webmail
  - ◆ 用户使用浏览器和HTTP协议与邮件服务器通信，收发邮件
  - ◆ 邮件服务器之间使用SMTP发送邮件
- 安全性：PGP, SMIME

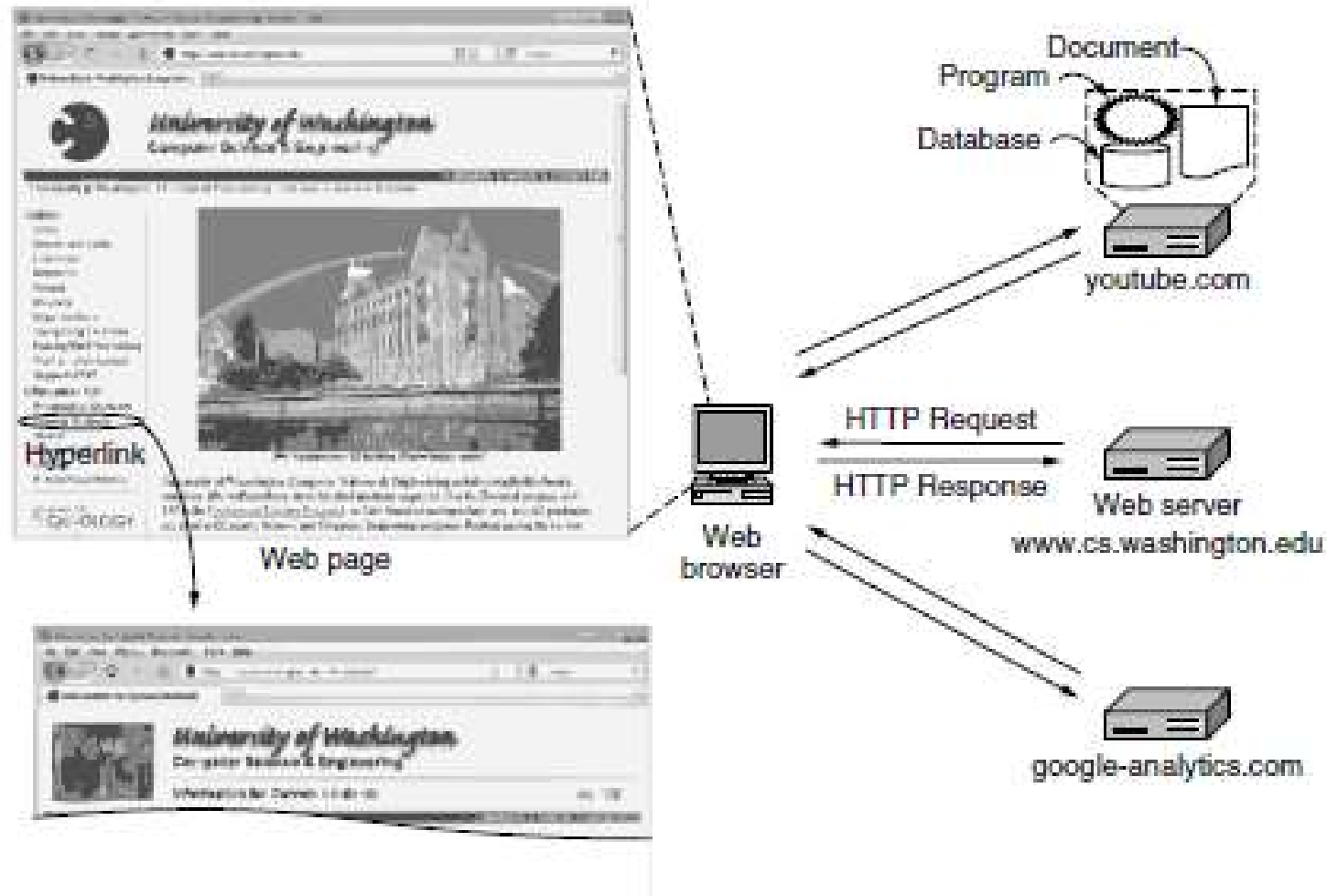
# Outline

- Introduction to Application Layer
- Domain Name System (DNS)
- Email System and Protocols
- World Wide Web (WWW) (Textbook 7.3)
  - ◆ WWW Architecture
  - ◆ Hypertext Transfer Protocol (HTTP)

# WWW: World Wide Web

- One of mostly widely used Internet application
- Not a network, but an "universe of network-accessible information, an embodiment of human knowledge" (*Tim Berners-Lee* )
- A distributed hyper-media system, including multiple media information such as text, graph, image, audio, even video
- Information is distributed in Internet, stored in **web servers** and accessed by **hyperlinks**(超级链接)
- Client/Server mode, with **browser** program as client
- Client and server interact with **HTTP**: Hyper-Text Transfer Protocol(超文本传送协议)
- WWW documents are located with **URL**: Uniform Resource Locator(统一资源定位符)
- WWW documents are written in **HTML**: HyperText Markup Language(超文本标记语言)
- Searching engines facilitate user to find information needed

# Architectural Overview of WWW



## Client Side: Browser(浏览器)

- An application program, as user's interface to Web
- Fetching information (web page) from Web server
- Displaying information for users
- After the user clicks on a hyperlink, **Browser**
  - ◆ Determines URL, name of the linked web page, eg. [www.abc.com/example.html](http://www.abc.com/example.html)
  - ◆ Asks **DNS** for the IP address of [www.abc.com](http://www.abc.com)
  - ◆ Establishes a TCP connection to replied IP address
  - ◆ Sends over an HTTP request asking for the page [/example.html](http://www.abc.com/example.html)
  - ◆ Fetches embedded images
  - ◆ Displays all the texts and images in [/example.html](http://www.abc.com/example.html)
  - ◆ Releases TCP connection



# Web Server

- Stores set of Web documents
- Responds to request from browser by sending copy of document
- Steps of server's loop
  - ◆ Accept a TCP connection from a client (a browser).
  - ◆ Get the path to the page, which is the name of the file requested.
  - ◆ Get the file (from disk).
  - ◆ Send the contents of the file to the client.
  - ◆ Release the TCP connection

# Web Documents

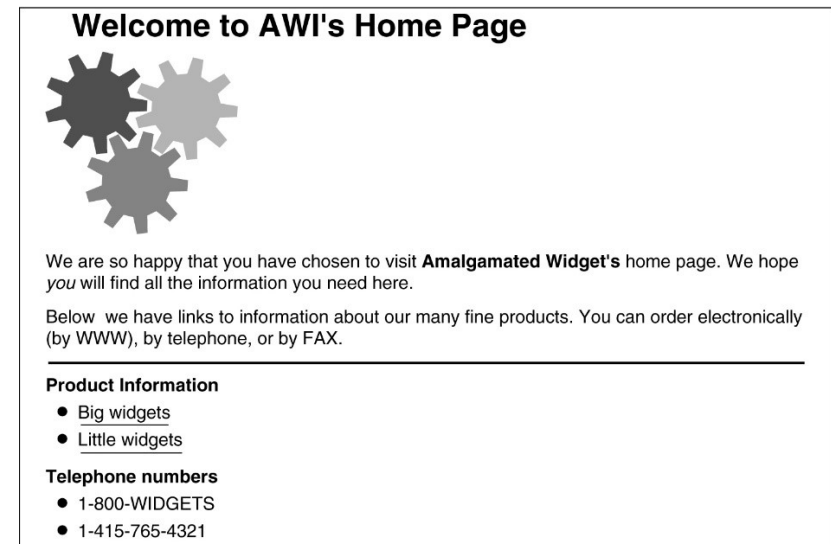
- Also called a web page
- Types
  - ◆ Static web page
    - Stored in file
    - Unchanging
  - ◆ Dynamic web page
    - Can **change** whenever a server receives a request
    - Generated on **server side**
    - Output from a program on server, usually a script
  - ◆ Active web page
    - Can **change** after the document has been loaded into a browser
    - Generated on **client side**
    - Consists of a computer program, known as Java, running in browser
    - Document called applet

# HTML – 超文本标记语言

```
<html>
<head><title> AMALGAMATED WIDGET, INC. </title> </head>
<body> <h1> Welcome to AWI's Home Page</h1>
 <br>
We are so happy that you have chosen to visit <b> Amalgamated Widget's </b>
home page. We hope<i> you </i> will find all the information you need here.
<p>Below we have links to information about our many fine products.
You can order electronically (by WWW), by telephone, or by fax. </p>
<hr>
<h2> Product information </h2>
<ul>
  <li> <a href="http://widget.com/products/big"> Big widgets </a>
  <li> <a href="http://widget.com/products/little"> Little widgets </a>
</ul>
<h2> Telephone numbers</h2>
<ul>
  <li> By telephone: 1-800-WIDGETS
  <li> By fax: 1-415-765-4321
</ul>
</body>
</html>
```

(a)

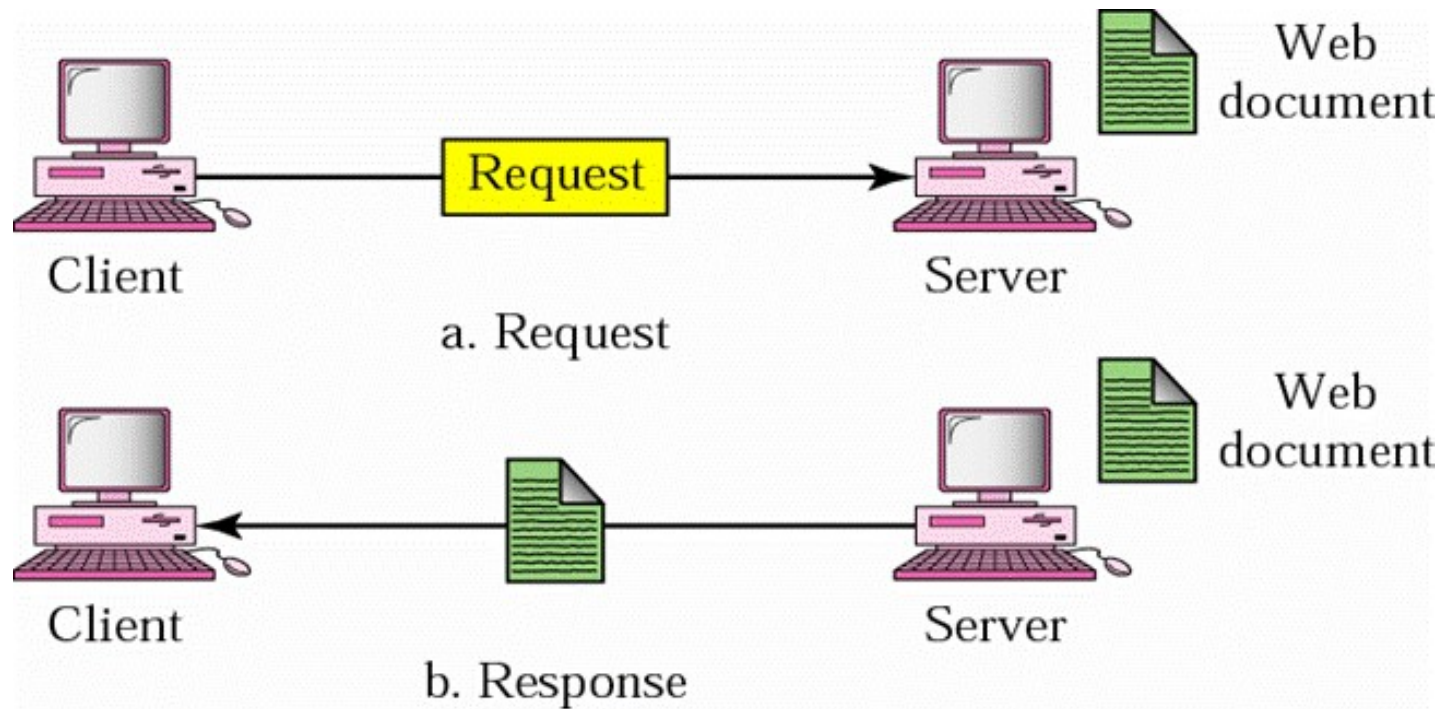
(a) 一个样例网页的超文本标记代码.



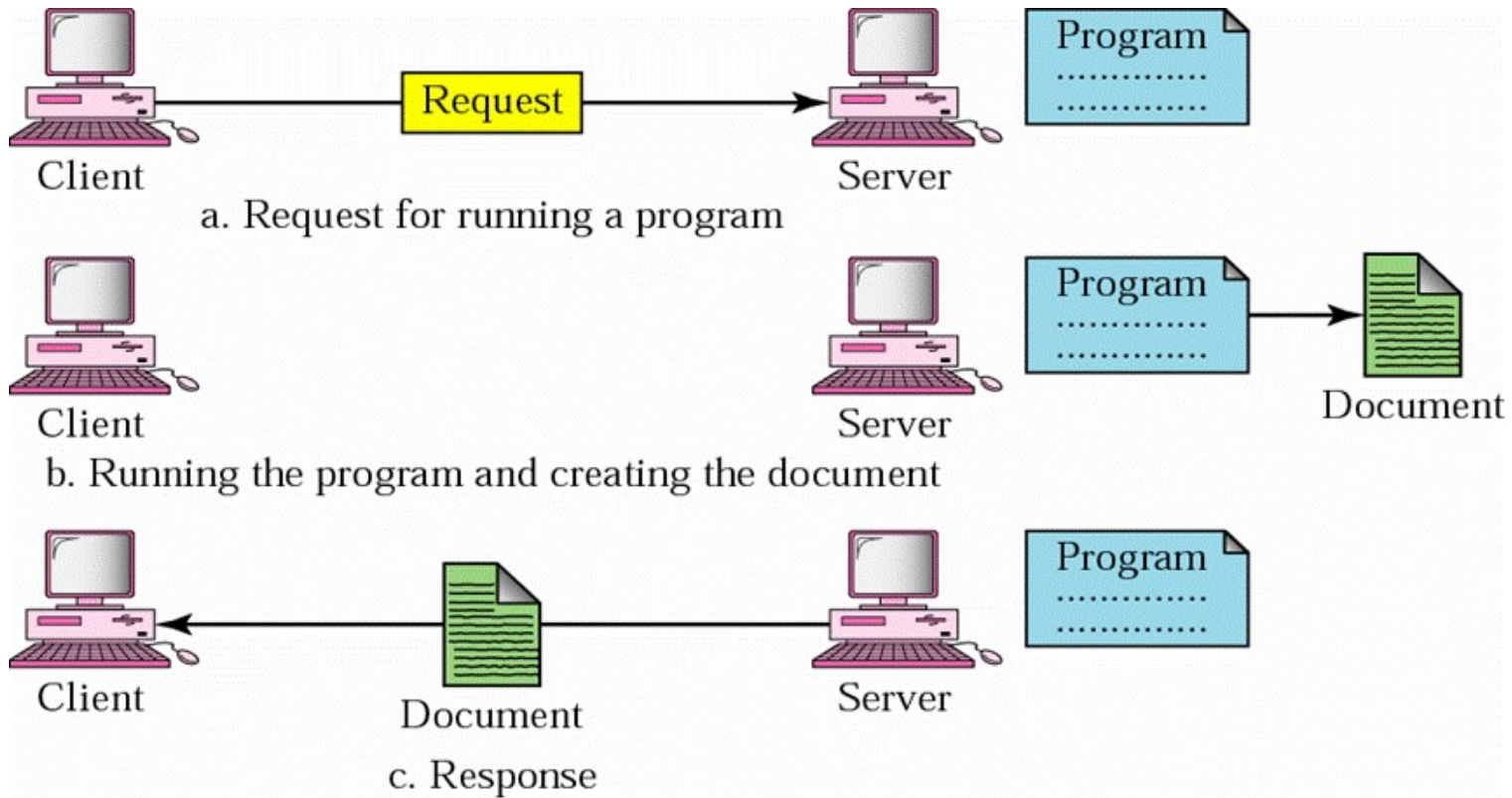
(b)

(b) 格式化网页.

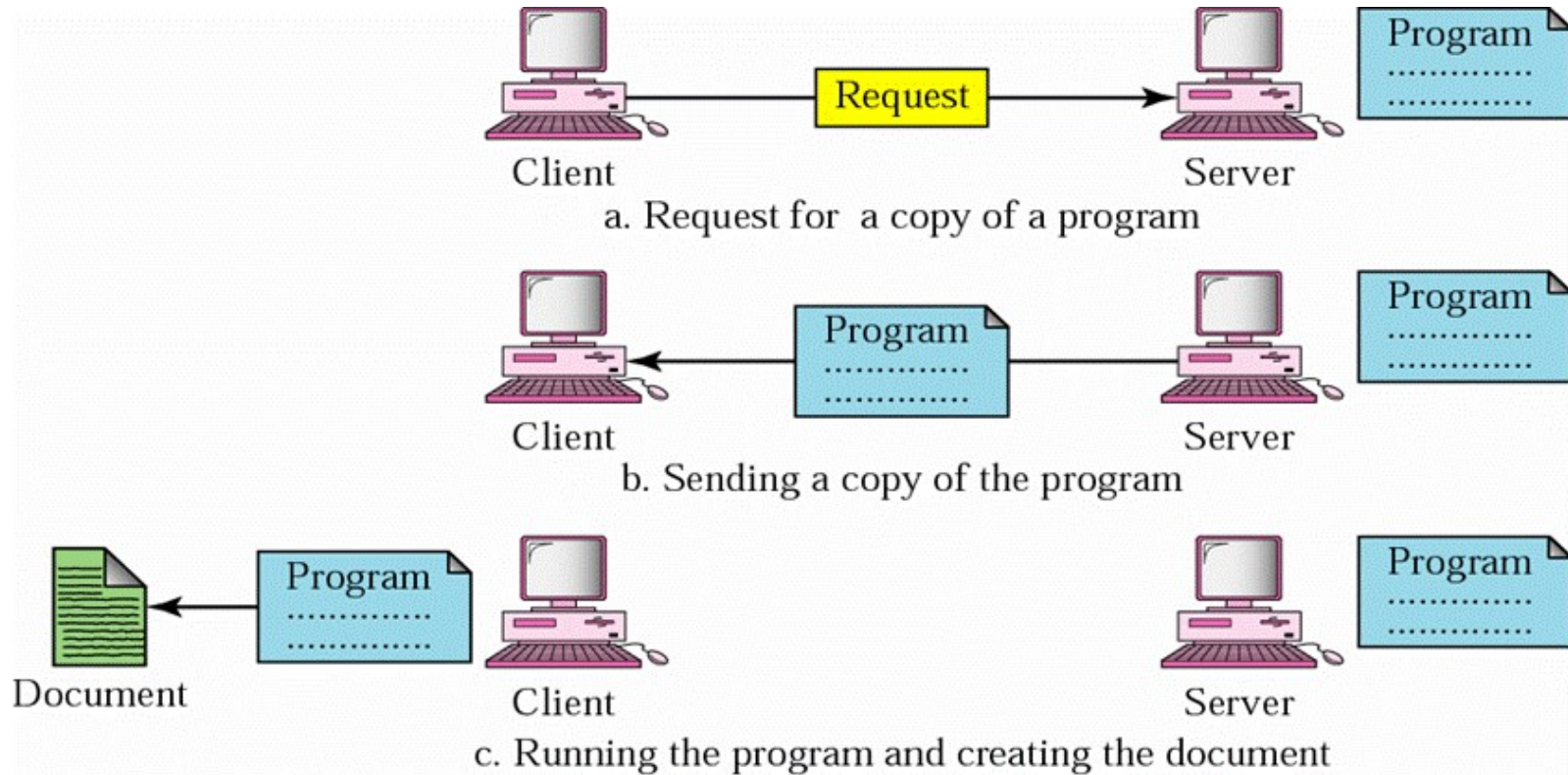
# Static web pages



# Dynamic web pages



# Active Web pages





# Links to Other Pages

- Embedded in HTML document
- Browser
  - ◆ Hides text of link from user
  - ◆ Associates link with item on page
- Called Uniform Resource Locator (**URL**)
  - ◆ Serving as worldwide name of a web page
  - ◆ By default, Protocol is *http* ; Port is *80*; Path is *index.html*

`protocol` :// `domain_name` : `port` / `item_name`

name of access protocol to use      domain name of server computer      protocol port number      path name of item

## Some Common URLs

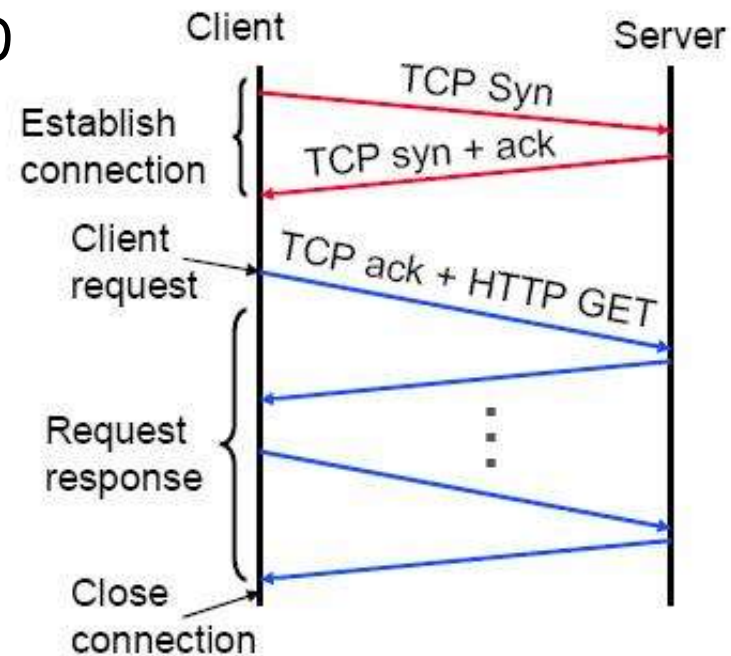
- Web browsers are often called Universal Clients because most can talk other protocols besides HTTP

| Name   | Used for                | Example                                                                               |
|--------|-------------------------|---------------------------------------------------------------------------------------|
| http   | Hypertext (HTML)        | <a href="http://www.ee.uwa.edu/~rob/">http://www.ee.uwa.edu/~rob/</a>                 |
| https  | Hypertext with security | <a href="https://www.bank.com/accounts/">https://www.bank.com/accounts/</a>           |
| ftp    | FTP                     | <a href="ftp://ftp.cs.vu.nl/pub/minix/README">ftp://ftp.cs.vu.nl/pub/minix/README</a> |
| file   | Local file              | <a href="file:///usr/suzanne/prog.c">file:///usr/suzanne/prog.c</a>                   |
| mailto | Sending email           | <a href="mailto:JohnUser@acm.org">mailto:JohnUser@acm.org</a>                         |
| rtsp   | Streaming media         | <a href="rtsp://youtube.com/montypython.mpg">rtsp://youtube.com/montypython.mpg</a>   |
| sip    | Multimedia calls        | <a href="sip:eve@adversary.com">sip:eve@adversary.com</a>                             |
| about  | Browser information     | <a href="about:plugins">about:plugins</a>                                             |

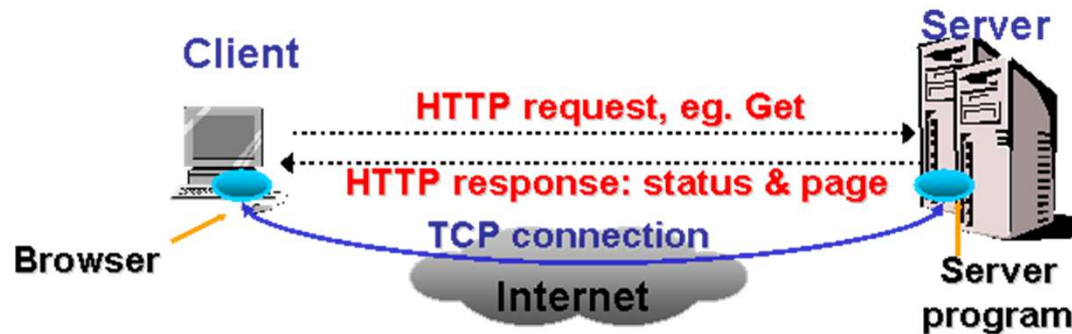


# HTTP : Hyper-Text Transfer Protocol

- Client-server architecture
- Synchronous **request/reply** protocol
  - ◆ Running over TCP, Port 80
- Stateless
  - ◆ Each command is executed **independently**, without any knowledge of the previous operations

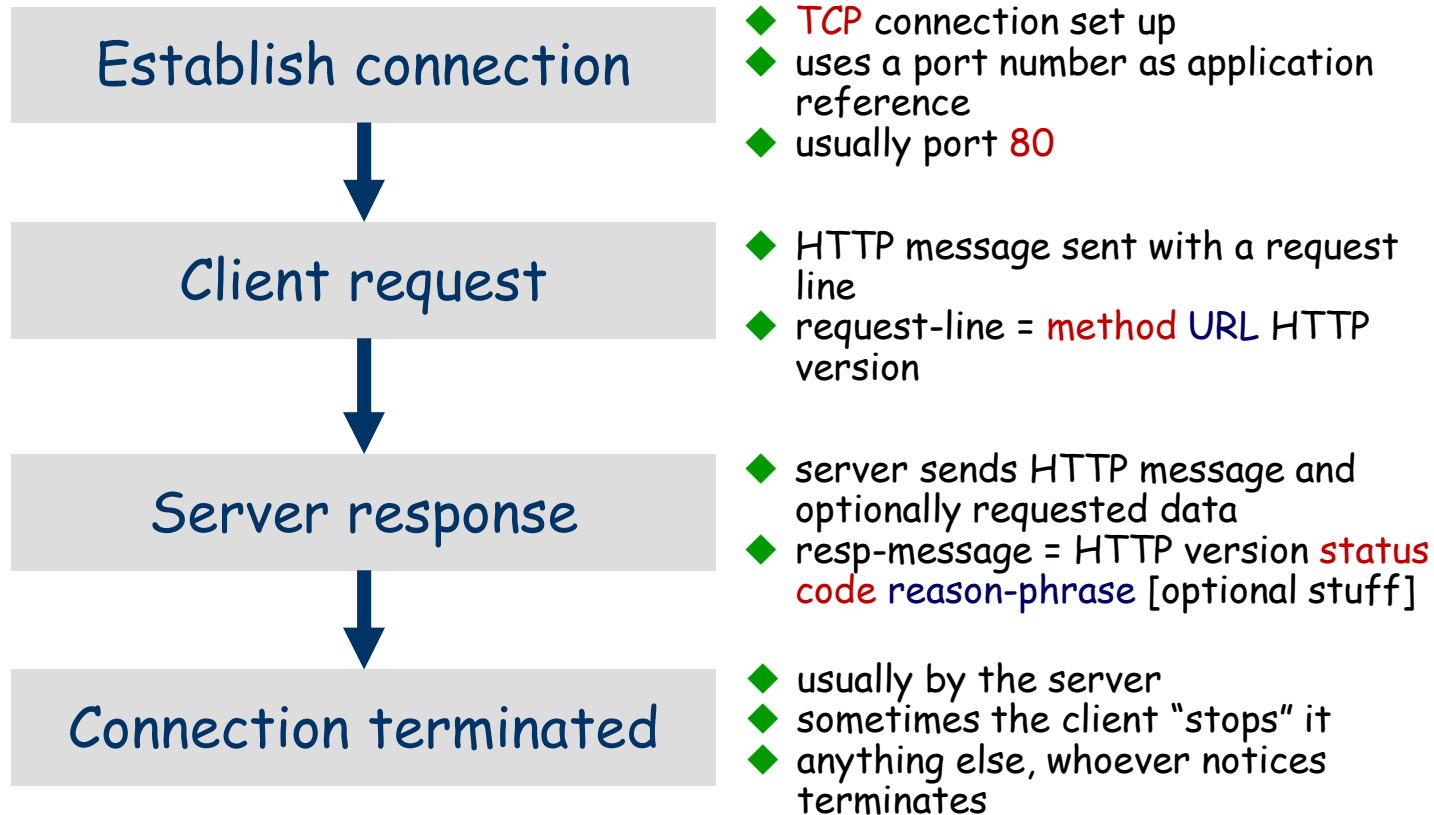


# Operations of HTTP



- (1) Browser analyses URL of the hyperlink, i.e.  
`http://www.abc.com/example.html`
- (2) Browser uses DNS to get IP address of the server  
(`www.abc.com`)
- (3) Browser sets up a TCP connection with server
- (4) Browser sends a HTTP request: **GET** /example.html **HTTP/1.0**
- (5) Server sends requested page to browser
- (6) TCP connection is released
- (7) Browser displays `example.html`

# HTTP Transaction



# HTTP Methods

| Method  | Description                                   |
|---------|-----------------------------------------------|
| GET     | Request to read a Web page                    |
| HEAD    | Request to read a Web page's header           |
| PUT     | Request to store a Web page                   |
| POST    | Append to a named resource (e.g., a Web page) |
| DELETE  | Remove the Web page                           |
| TRACE   | Echo the incoming request                     |
| CONNECT | Reserved for future use                       |
| OPTIONS | Query certain options                         |

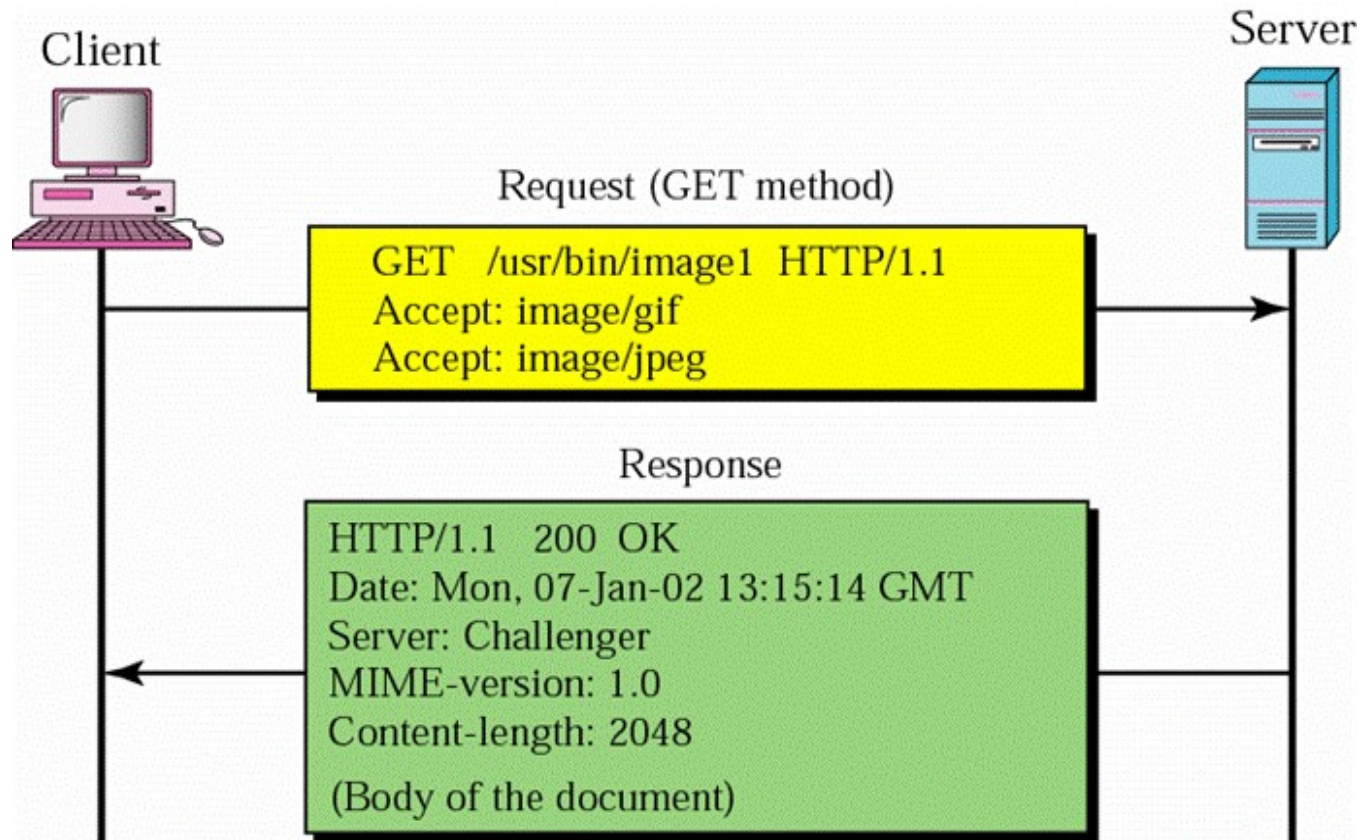
# HTTP Message Headers

| Header           | Type     | Contents                                              |
|------------------|----------|-------------------------------------------------------|
| User-Agent       | Request  | Information about the browser and its platform        |
| Accept           | Request  | The type of pages the client can handle               |
| Accept-Charset   | Request  | The character sets that are acceptable to the client  |
| Accept-Encoding  | Request  | The page encodings the client can handle              |
| Accept-Language  | Request  | The natural languages the client can handle           |
| Host             | Request  | The server's DNS name                                 |
| Authorization    | Request  | A list of the client's credentials                    |
| Cookie           | Request  | Sends a previously set cookie back to the server      |
| Date             | Both     | Date and time the message was sent                    |
| Upgrade          | Both     | The protocol the sender wants to switch to            |
| Server           | Response | Information about the server                          |
| Content-Encoding | Response | How the content is encoded (e.g., gzip)               |
| Content-Language | Response | The natural language used in the page                 |
| Content-Length   | Response | The page's length in bytes                            |
| Content-Type     | Response | The page's MIME type                                  |
| Last-Modified    | Response | Time and date the page was last changed               |
| Location         | Response | A command to the client to send its request elsewhere |
| Accept-Ranges    | Response | The server will accept byte range requests            |
| Set-Cookie       | Response | The server wants the client to save a cookie          |

# HTTP Status Codes

| Code | Meaning      | Examples                                           |
|------|--------------|----------------------------------------------------|
| 1xx  | Information  | 100 = server agrees to handle client's request     |
| 2xx  | Success      | 200 = request succeeded; 204 = no content present  |
| 3xx  | Redirection  | 301 = page moved; 304 = cached page still valid    |
| 4xx  | Client error | 403 = forbidden page; 404 = page not found         |
| 5xx  | Server error | 500 = internal server error; 503 = try again later |

# Example of HTTP Transaction



*[Forouzan]*



# HTTP/1.1

## Performance Enhancements

- HTTP/1.0 is a “stop and wait” protocol
  - ◆ Separate TCP connection for each file
    - Connect setup and tear down is incurred for each file
    - Inefficient use of packets
    - Server must maintain many connections
- HTTP/1.1 specification focus on performance enhancements
  - ◆ Persistent connections
  - ◆ Pipelining
  - ◆ Enhanced caching options
  - ◆ Support for compression

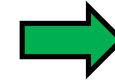


# HTTP/1.1

## Persistent Connections and Pipelining

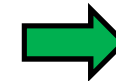
### ■ Persistent connections(持续连接)

- ◆ Use the same TCP connection(s) for transfer of multiple files
- ◆ Reduces packet traffic significantly

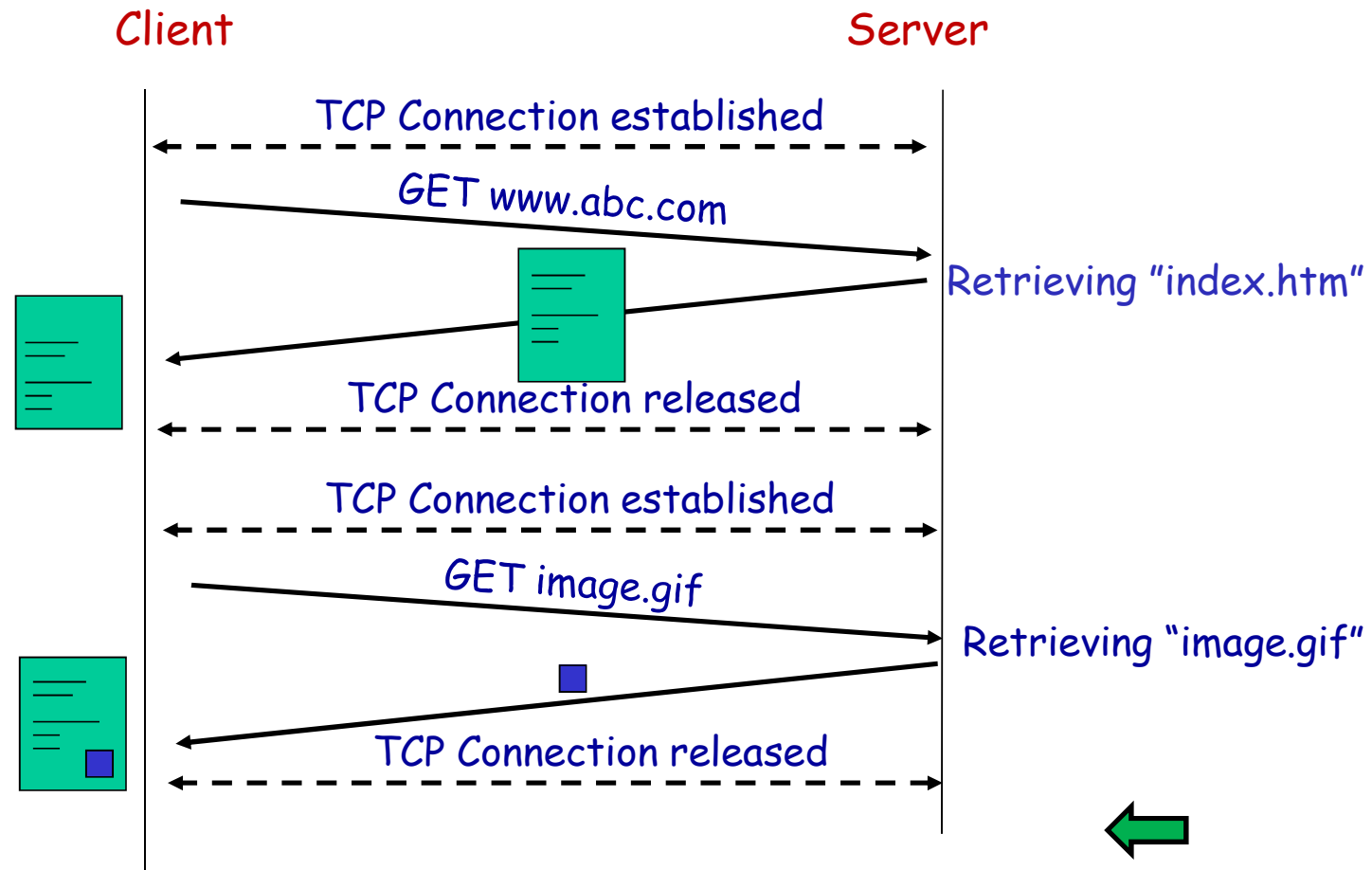


### ■ Pipelining(流水线)

- ◆ **Multiple HTTP requests** can be written out to a socket together without waiting for the corresponding responses.
- ◆ Pack several HTTP requests into one TCP/IP packet

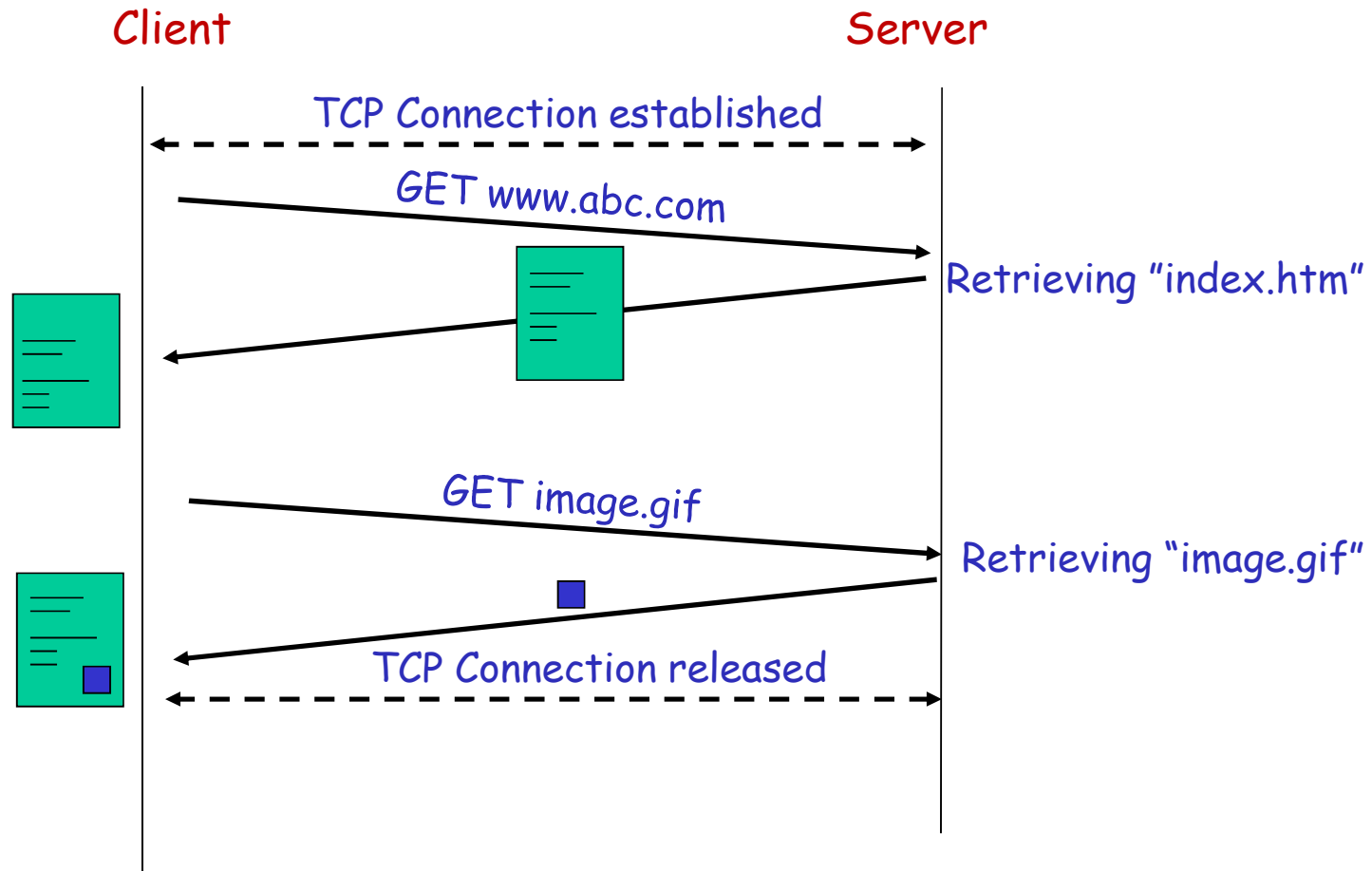


# Example of Non-persistent connections

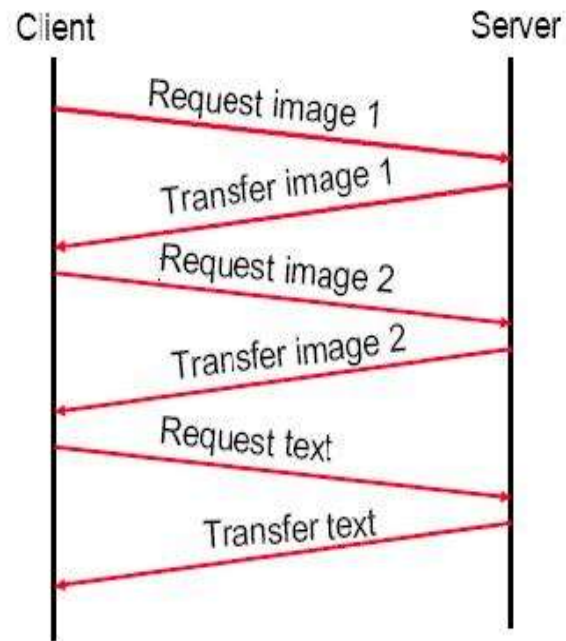


[Kurose]

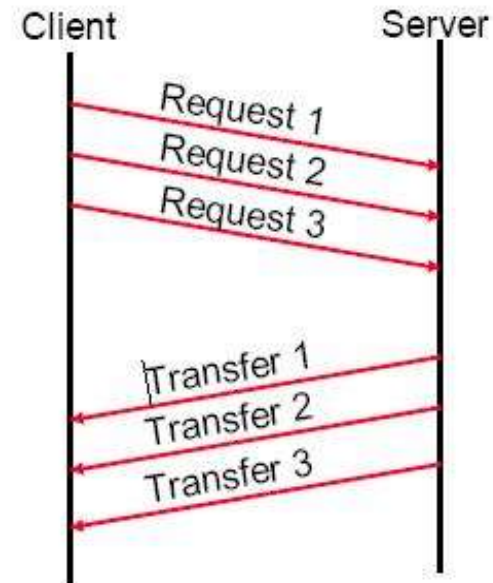
# Example of Persistent connections



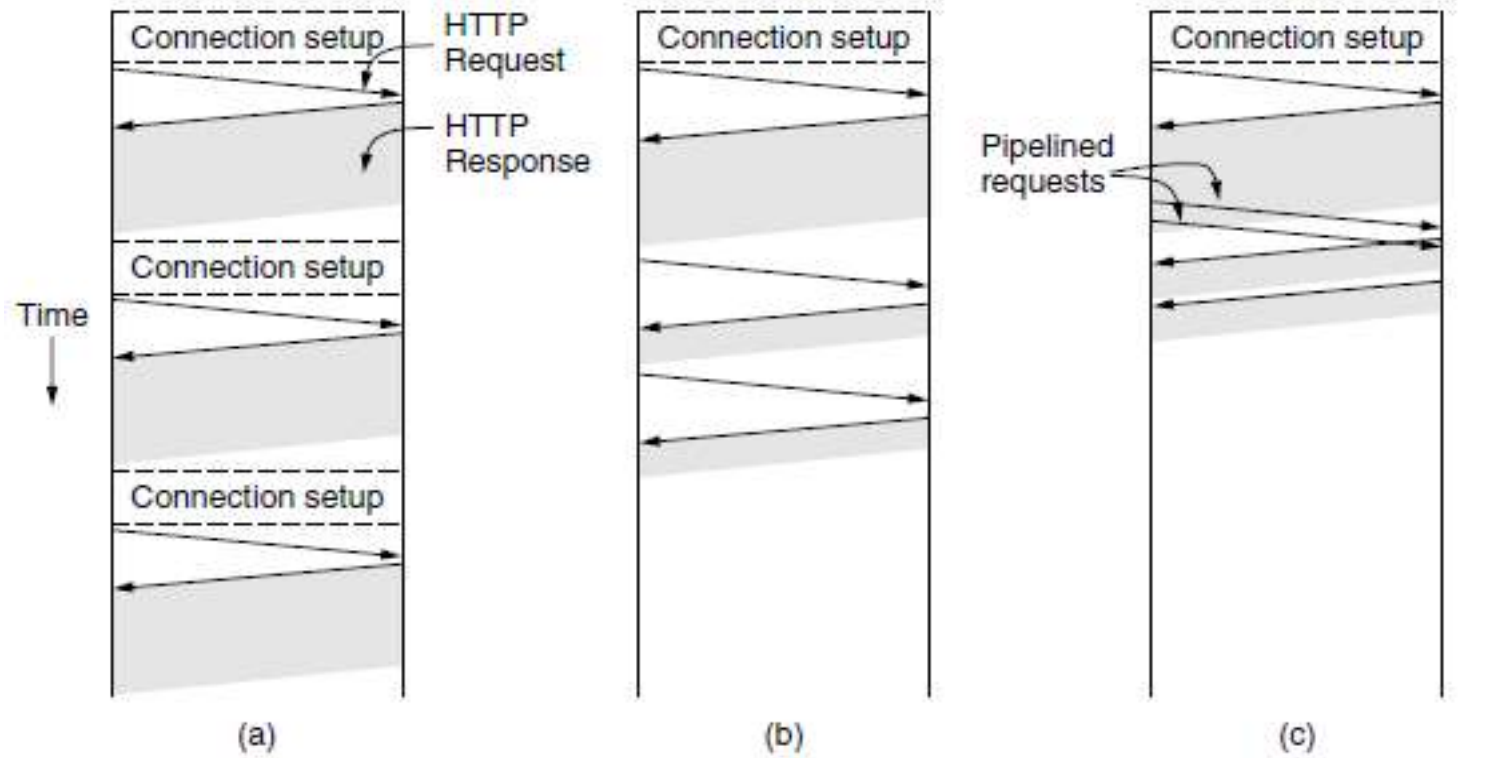
## Example of Pipelining(流水线)



Non-pipelining



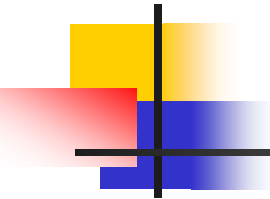
Pipelining



multiple connections  
and sequential requests

A persistent  
connection and  
sequential requests

A persistent  
connection and  
pipelined requests



# User-server state: cookies

---

- Many major Web sites use cookies

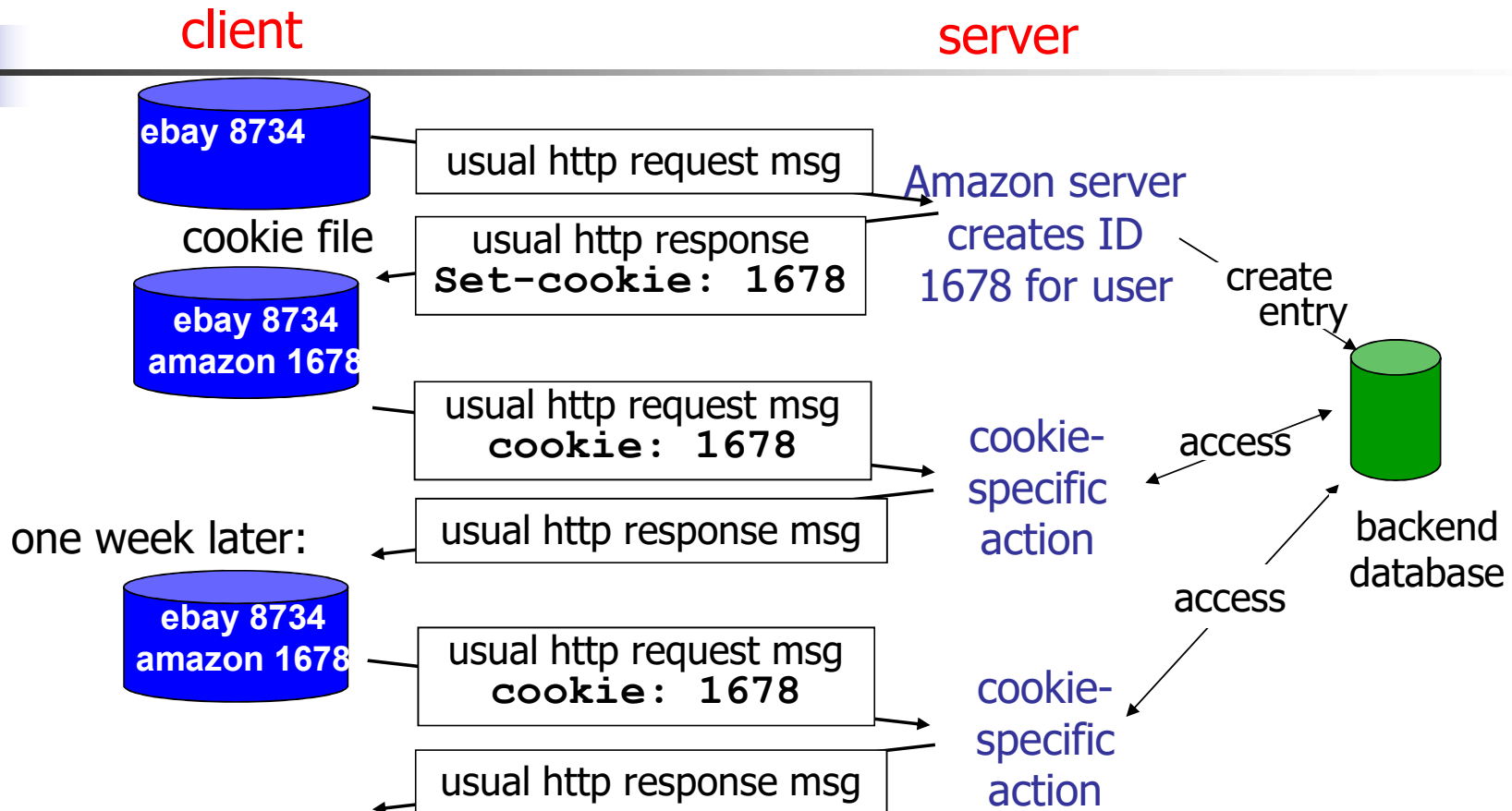
## Example:

- Susan always access Internet from PC
- She visits specific e-commerce site for first time
- when initial HTTP request arrives at site, site creates:
  - unique ID
  - entry in backend database for ID

## Four components:

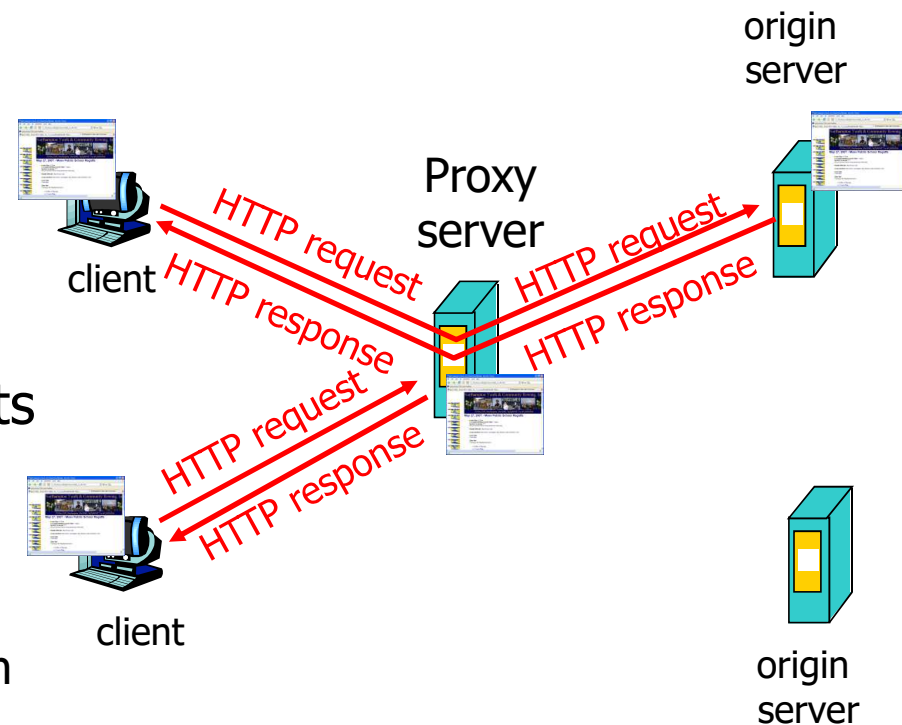
- 1) cookie header line of HTTP *response* message
- 2) cookie header line in HTTP *request* message
- 3) cookie file kept on *user's host*, managed by user's browser
- 4) back-end *database at Web site*

# Cookies: keeping "state"



# Web Caches (Proxy Server)

- Motivation: satisfy client request without involving origin server
- User sets browser: Web accesses via a **proxy server**
- Browser sends all HTTP requests to proxy server
  - If requested file in cache: proxy server returns file
  - else proxy requests file from origin server, then forwards to client





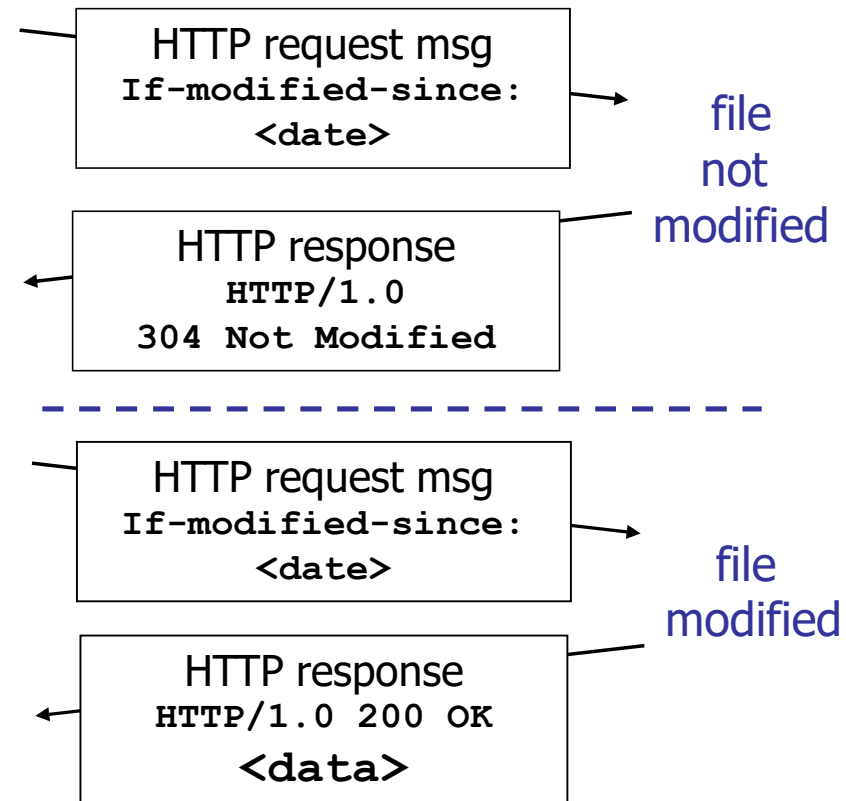
# Conditional get

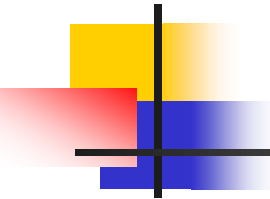
- Server does not send required files if cache has up-to-date cached version
- cache: specify date of cached copy in HTTP request  
`If-modified-since: <date>`
- server: response contains no object if cached copy is up-to-date:

`HTTP/1.0 304 Not Modified`

cache

server





# HTTPS

---

- Hypertext Transfer Protocol Secure (HTTPS)
- HTTP on top of the SSL(Secure Sockets Layer)/TLS (Transport Layer Security) protocol
- A communication protocol for secure communication over a computer network, with especially wide deployment on the Internet

## WWW小结

- 目标：将分布在互联网不同设备上的资源组织起来。
- WWW体系结构
  - ◆ 超链接：超文本、超媒体
  - ◆ Web客户(浏览器)和Web服务器
  - ◆ URL: `protocol://domain_name:port/item_name`
- Web页面：静态页面(HTML)，动态页面，活动页面
- 超文本传输协议(HTTP)
  - ◆ Client/Server, 同步数据传输
  - ◆ over TCP, port 80
  - ◆ HTTP操作：TCP连接，HTTP请求报文和响应报文
  - ◆ HTTP1.0&1.1: 持久连接、流水线请求

## Key Points(1)

- Idea of client/server model
- DNS
  - ◆ Domain name, hierarchical architecture
  - ◆ Jobs of resolver and DNS servers
  - ◆ Iterative resolution vs. Recursive resolution
- WWW
  - ◆ Concepts of WWW, HTTP, HTML, hyperlink, URL, browser
  - ◆ Operations of HTTP
  - ◆ HTTP 1.0 vs. HTTP 1.1
  - ◆ Function of cookies, proxy and conditional get

## Key Points(2)

### ■ Email

- ◆ Email address
- ◆ SMTP: sending Email from user machine to receiver's mailbox
- ◆ POP3/IMAP: downloading Email from mailbox to user machine
- ◆ Web mail: using HTTP

# Terms

| 缩写   | 英文全称                                 | 中文        |
|------|--------------------------------------|-----------|
|      | Application Layer                    | 应用层       |
|      | Authoritative Name Server            | 权威名字服务器   |
|      | browser                              | 浏览器       |
|      | distributed database                 | 分布式数据库    |
|      | domain                               | 域         |
| DNS  | Domain Name System                   | 域名系统      |
|      | hyperlink                            | 超链接       |
| HTML | HyperText Markup Language            | 超文本标记语言   |
| HTTP | HyperText Transfer Protocol          | 超文本传送协议   |
| IMAP | Internet Message Access Protocol     | 因特网报文访问协议 |
|      | Internet Message Format              | 因特网报文格式   |
|      | Iterative resolution                 | 迭代查询      |
| MIME | Multipurpose Internet Mail Extension | 多用途邮件扩展   |
| MTA  | Mail Transfer Agent                  | 邮件传送代理    |

| 缩写   | 英文全称                          | 中文       |
|------|-------------------------------|----------|
|      | Name Server                   | 名字服务器    |
|      | Name space                    | 名字空间     |
|      | Non-Persistent connections    | 非持久连接    |
| P2P  | Peer-to-peer                  | 对等体      |
|      | Persistent connections        | 持久连接     |
|      | pipelining                    | 流水线      |
| POP  | Post Office Protocol          | 邮局协议     |
|      | proxy                         | 代理       |
|      | Recursive resolution          | 递归解析     |
|      | resolver                      | 解析器      |
|      | Resource records              | 资源记录     |
|      | searching engine              | 搜索引擎     |
| SMTP | Simple Mail Transfer Protocol | 简单邮件传送协议 |

| 缩写   | 英文全称                          | 中文       |
|------|-------------------------------|----------|
| SMTP | Simple Mail Transfer Protocol | 简单邮件传送协议 |
|      | User Agent                    | 用户代理     |
| URL  | Uniform Resource Locator      | 统一资源定位符  |
| WWW  | World Wide Web                | 万维网      |
|      | zone                          | 区        |



# Copyright Information

- Some figures used in this presentation have been either directly copied or adapted from following materials
  - ◆ “Data communications and networking”, B.A.Forouzan, which are marked with [Forouzan]
  - ◆ Lecture notes of book “Computer Networks, a Top-down Approach”, J.Kurose and W.Ross, 3<sup>rd</sup> Edition, which are marked with [Kurose]
  - ◆ “计算机网络”, 谢希仁, 第五版, 电子工业出版社, 2008年, which are marked with [谢]